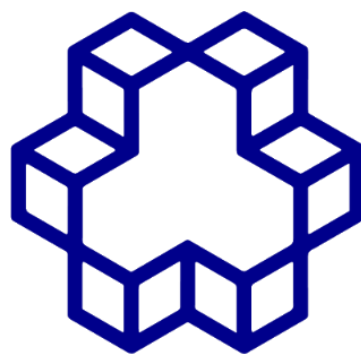


بسم الله الرحمن الرحيم



دانشگاه صنعتی خواجه نصیرالدین طوسی

یادگیرر تقوی

پروژه پایانی

امیر محمد حامد

۴۰۴۰۰۳۵۶

۱۴۰۵/۰۵/۱۶

چکیده

در این پروژه، یک محیط هزارتوی دوبعدی و تصادفی طراحی شد که عامل باید ابتدا کلید را بردارد، سپس از در عبور کند و در نهایت به هدف برسد. براساس شماره دانشجویی ۴۰۴۰۰۳۵۶، مقدار Seed پایه برابر با ۵ و اندازه نقشه ۱۶×۱۶ در نظر گرفته شد. حالت عامل به صورت (row, column, has_key) تعریف شد تا داشتن یا نداشتن کلید نیز به طور مستقیم در تصمیم‌گیری عامل لحاظ شود. عامل می‌تواند یکی از چهار عمل بالا، پایین، چپ و راست را انتخاب کند. عمل انتخاب‌شده با احتمال ۰.۸ اجرا می‌شود و هر یک از دو انحراف عمود بر مسیر نیز با احتمال ۰.۱ رخ می‌دهد. علاوه بر این، یک جفت انتقال‌دهنده دوطرفه به محیط اضافه شد که برخلاف یک نشانه صرفاً گرافیکی، بخشی از تابع انتقال واقعی محیط است.

سه الگوریتم Value Iteration، Q-Learning و $SARSA(\lambda)$ بدون استفاده از پیاده‌سازی‌های آماده یادگیری تقویتی توسعه داده شدند. ساختار پاداش نیز در دو حالت sparse و potential-based shaping بررسی شد. آزمایش اصلی با ۱۰ Seed مستقل انجام گرفت و در مجموع شامل ۷۱ اجرای مبدأ، ۱۲۰ اجرای انتقال، ۲۱۰۰۰۰ ردیف اپیزود آموزشی در محیط مبدأ و ۱۸۰۰۰۰ ردیف اپیزود آموزشی در محیط مقصد بود.

الگوریتم Value Iteration برای سه مقدار ۷ برابر با ۰.۸۰، ۰.۹۵ و ۰.۹۹ به همگرایی رسید و بیشترین Bellman residual ثبت‌شده برابر با ۶.۷۲×۱۰^{-۸۱} بود. در Q-Learning، کاهش نمایی ϵ باعث شد عامل سریع‌تر به آستانه موفقیت برسد، در حالی که کاهش خطی ϵ سازگاری بیشتری میان سیاست آموخته‌شده و سیاست مرجع ایجاد کرد. در $SARSA$ نیز $\lambda=۰.۹$ سریع‌ترین عبور از آستانه موفقیت را به همراه داشت، اما نوسان زیاد و کاهش بازده در مرحله ارزیابی مشاهده شد. به همین دلیل، $\lambda=۰.۷$ به عنوان گزینه‌ای متعادل‌تر انتخاب شد.

در بخش انتقال یادگیری، انتقال کامل و انتقال ضریب‌دار در محیط مشابه، از همان ابتدای کار به نرخ موفقیت ۱.۰ رسیدند. در محیط متفاوت، نرخ موفقیت اولیه این دو روش برابر با ۰.۷۰۴ ± ۰.۰۱۴ بود؛ در مقابل، آموزش از صفر و انتقال انتخابی در ارزیابی اولیه موفقیتی نداشتند. پس از انجام آموزش در محیط مقصد، نرخ موفقیت همه سناریوها به ۱.۰ رسید، اما همچنان تفاوت‌هایی از نظر تعداد نمونه موردنیاز، بازده و میزان توافق سیاست‌ها وجود داشت.

همچنین، دو نمونه از انتقال منفی با مقادیر دقیق regret برابر با ۶.۱۷۸ و ۱۰.۸۳۵ شناسایی شد. در هر دو نمونه، فرایند آموزش در محیط مقصد توانست عمل نامناسبی را که از محیط مبدأ منتقل شده بود اصلاح کند. در کنار ارزیابی الگوریتم‌ها، رابط گرافیکی مستقل، مدل‌های ذخیره‌شده، ۱۴ نمودار، ۱۹۶ آرشیو مدل و ۴۵ تست واحد و یکپارچه نیز بررسی شدند.

نتایج نشان می‌دهد که نرخ موفقیت نهایی به‌تنهایی معیار کافی برای مقایسه روش‌های یادگیری نیست. برای ارزیابی دقیق‌تر، لازم است سرعت یادگیری، پایداری فرایند آموزش، کیفیت سیاست نهایی و خطاهای موضعی نیز به‌صورت هم‌زمان در نظر گرفته و گزارش شوند.

فهرست مطالب

۱- مقدمه	۷
۱-۱- هدف پروژه	۷
۲-۱- خروجی‌های اصلی	۷
۲- تعریف مسئله و طراحی محیط	۸
۲-۱- Seed های اختصاصی و نقشه	۸
۲-۲- اعتبارسنجی مسیر با BFS	۹
۳-۲- انتقال‌دهنده دوطرفه	۹
۳- مدلسازی فرایند تصمیم‌گیری مارکوف	۱۰
۳-۱- اجزای فرایند تصمیم‌گیری مارکوف	۱۰
۳-۲- اندازه و دسترسی‌پذیری فضای حالت	۱۰
۳-۳- مدل انتقال تصادفی	۱۰
۴-۳- موفقیت در پایان و محدودیت گام	۱۱
۴- طراحی تابع پاداش و ثبت رویدادها	۱۲
۴-۱- پاداش Sparse	۱۲
۴-۲- پاداش پتانسیل مبنا	۱۲
۴-۳- رویدادهای اجباری و نمونه واقعی	۱۲
۴-۴- بررسی رفتار ناخواسته	۱۳
۵- ساختار پیاده‌سازی و پروتکل آزمایش	۱۴
۵-۱- ساختار ماژول‌ها	۱۴
۵-۲- تنظیمات مشترک	۱۴
۵-۳- طراحی تصادفی‌سازی و ارزیابی	۱۴
۵-۴- اندازه ماتریس آزمایش	۱۵
۶- الگوریتم Value Iteration	۱۶
۶-۱- پیاده‌سازی و به‌روزرسانی بلمن	۱۶
۶-۲- حساسیت به ضریب کاهش	۱۶
۶-۳- نقشه حرارتی و سیاست بهینه	۱۷
۷- الگوریتم Q-Learning	۱۹

۱۹	۷-۱- به روزرسانی off-policy
۱۹	۷-۲- برنامه های کاهش Epsilon
۲۰	۷-۳- بازسازی یک Q-update واقعی
۲۰	۷-۴- مقایسه پاداش های Shaped و Sparse
۲۱	۷-۵- بازدید حالت و مسیر نهایی
۲۳	۸- الگوریتم $SARSA(\lambda)$
۲۳	۸-۱- به روزرسانی on-policy و Eligibility Traces
۲۳	۸-۲- تبدیل به SARSA یک مرحله ای در $\lambda=0$
۲۳	۸-۳- بازسازی چند گام واقعی از E و δ
۲۴	۸-۴- تحلیل حساسیت λ
۲۵	۹- مقایسه سه الگوریتم
۲۵	۹-۱- مقایسه عددی در شرایط یکسان
۲۶	۹-۲- رفتاری on-policy و off-policy در نزدیکی خطر
۲۶	۹-۳- اختلاف سیاست با مرجع
۲۸	۱۰- انتقال یادگیری روی Q-Learning
۲۸	۱۰-۱- طراحی دو محیط مقصد
۲۸	۱۰-۲- چهار روش انتقال و شش سناریوی عملی
۲۹	۱۰-۳- نتایج مقصد مشابه
۲۹	۱۰-۴- نتایج متضد متفاوت
۳۰	۱۰-۵- منحنی یادگیری و اثر β
۳۲	۱۰-۶- انتقال منفی و اصلاح رفتار
۳۳	۱۱- رابط گرافیکی و خروجی های بصری
۳۳	۱۱-۱- قابلیت های رابط
۳۴	۱۱-۲- خروجی های تصویری خواسته شده
۳۵	۱۲- آزمون، صحت و بازتولیدپذیری
۳۵	۱۲-۱- آزمون های اجرا شده
۳۵	۱۲-۲- کنترل های بسته و داده
۳۵	۱۲-۳- دستورات بازتولید
۳۶	۱۳- پاسخ مستقیم به پرسش های تحلیلی صورت پروژه
۳۶	۱۳-۱- تعریف کامل MDP و حفظ خاصیت مارکوف

۱۳-۲- تفاوت on-policy و off-policy در رفتار واقعی.....	۳۶
۱۳-۳- نیاز Value Iteration به مدل و محدودیت دو رویکرد.....	۳۶
۱۳-۴- بهترین λ از نظر سرعت و پایداری.....	۳۶
۱۳-۵- سه حالت اختلاف سیاست.....	۳۷
۱۳-۶- مقایسه انتقال مشابه و متفاوت.....	۳۷
۱۴- محدودیت‌های مطالعه.....	۳۸
۱۴-۱- محدودیت محیط و مقیاس.....	۳۸
۱۴-۲- محدودیت آماری.....	۳۸
۱۴-۳- محدودیت معیارها.....	۳۸
۱۴-۴- محدودیت انتقال.....	۳۸
۱۵- جمع‌بندی.....	۳۹
۱۶- ساختار تحویل و الزامات مخزن.....	۴۰
پیوست.....	۴۱

فهرست شکل‌ها

شکل ۱-۱- خروجی تابع ارزش و سیاست کنترل بهینه در متلب. ERROR! BOOKMARK NOT DEFINED.

شکل ۱-۲- خروجی سیاست اغتشاش بدترین حالت در متلب. ERROR! BOOKMARK NOT DEFINED.

شکل ۱-۳- خروجی محاسبات نقطه زینی در متلب. ERROR! BOOKMARK NOT DEFINED.

شکل ۱-۴- شبیه‌سازی رو به جلو از شرط اولیه $x_1 = 3$ و بخش امتیازی $x_1 = 2.5$ ERROR! BOOKMARK NOT DEFINED.

شکل ۱-۵- کنترل و اغتشاش. ERROR! BOOKMARK NOT DEFINED.

شکل ۱-۶- نمودار حاصل از $x_1 = 3$ ERROR! BOOKMARK NOT DEFINED.

شکل ۱-۷- نمودار حاصل از $x_1 = 2.5$ ERROR! BOOKMARK NOT DEFINED.

شکل ۲-۱- نمودار اجزای MRDE. ERROR! BOOKMARK NOT DEFINED.

شکل ۲-۲- خروجی مقادیر ماندگار. ERROR! BOOKMARK NOT DEFINED.

شکل ۲-۳- نمودار رفتار در BACKWARD MRDE. ERROR! BOOKMARK NOT DEFINED.

شکل ۳-۱- نمودار موقعیت عرضی پهباده‌ها. ERROR! BOOKMARK NOT DEFINED.

شکل ۳-۲- محاسبه خطای عرضی پهباده‌ها. ERROR! BOOKMARK NOT DEFINED.

۱- مقدمه

۱-۱- هدف پروژه

هدف این پروژه صرفاً اجرای چند الگوریتم در یک محیط ساده GridWorld نبود بلکه مسئله به گونه ای طراحی شد که عامل برای تکمیل مأموریت، ابتدا کلید را بردارد، سپس از در عبور کند و در نهایت به هدف برسد. از طرف دیگر، به دلیل تصادفی بودن اجرای اعمال، یک سیاست مناسب نمی تواند تنها بر کوتاه ترین مسیر هندسی تکیه کند؛ بلکه باید احتمال انحراف از مسیر، برخورد با دیوار و ورود به خانه های دارای جریمه را نیز در تصمیم گیری خود در نظر بگیرد. این ویژگی ها محیط مناسبی را برای مقایسه یک روش مدل محور با دو روش مدل آزاد فراهم می کنند.

در روش مدل محور Value Iteration، الگوریتم به احتمال های انتقال و پاداش مربوط به تمام زوج های حالت-عمل دسترسی دارد. در مقابل، Q-Learning و SARSA(λ) مدل آزاد هستند؛ بنابراین این اطلاعات را به طور مستقیم دریافت نمی کنند و مقادیر عمل را براساس نمونه های واقعی جمع آوری شده در طول اپیزودها تخمین می زنند. برای آنکه تفاوت نتایج ناشی از تغییر شرایط محیط نباشد، هر سه الگوریتم روی یک نقشه مشترک و با ساختار پاداش و میزان تصادفی سازی یکسان ارزیابی شدند.

بخش انتقال یادگیری فقط برای الگوریتم Q-Learning اجرا شد. در این مرحله، جدول Q آموخته شده در نقشه مبدأ به دو نقشه مقصد با میزان تغییر متفاوت انتقال یافت. ارزیابی این بخش تنها به عملکرد نهایی محدود نشد؛ بلکه عملکرد اولیه پیش از آموزش در محیط مقصد، تعداد اپیزودها و نمونه های لازم برای رسیدن به آستانه موفقیت و همچنین موارد انتقال منفی نیز بررسی شدند. برای مشاهده حرکت مرحله به مرحله عامل و کنترل فرایند آموزش یا ارزیابی، یک رابط گرافیکی طراحی شد. تمام نمودارهای گزارش نیز براساس داده های خام ذخیره شده تولید شدند تا نتایج قابل بررسی و بازتولید باشند.

۱-۲- خروجی های اصلی

خروجی های پروژه مجموعه کاملی از نقشه های ذخیره شده، کد محیط و سه عامل، تنظیمات آزمایش، فایل های CSV اپیزودها، مدل های NPZ، شکل های تحلیلی، رابط گرافیکی و تست ها را دربر می گیرد. پروفایل نهایی با ۱۰ Seed اجرا شد و هیچ اجرای ناموفقی از مجموعه نتایج کنار گذاشته نشد. فایل run_manifest.json تعداد اجراها، هش کد، نسخه کتابخانه ها و هش خروجی های اصلی را ثبت می کند. به این ترتیب، هر عدد گزارش شده را می توان به اجرای مشخص و نسخه معینی از کد نسبت داد.

۲- تعریف مسئله و طراحی محیط

۲-۱- Seed های اختصاصی و نقشه

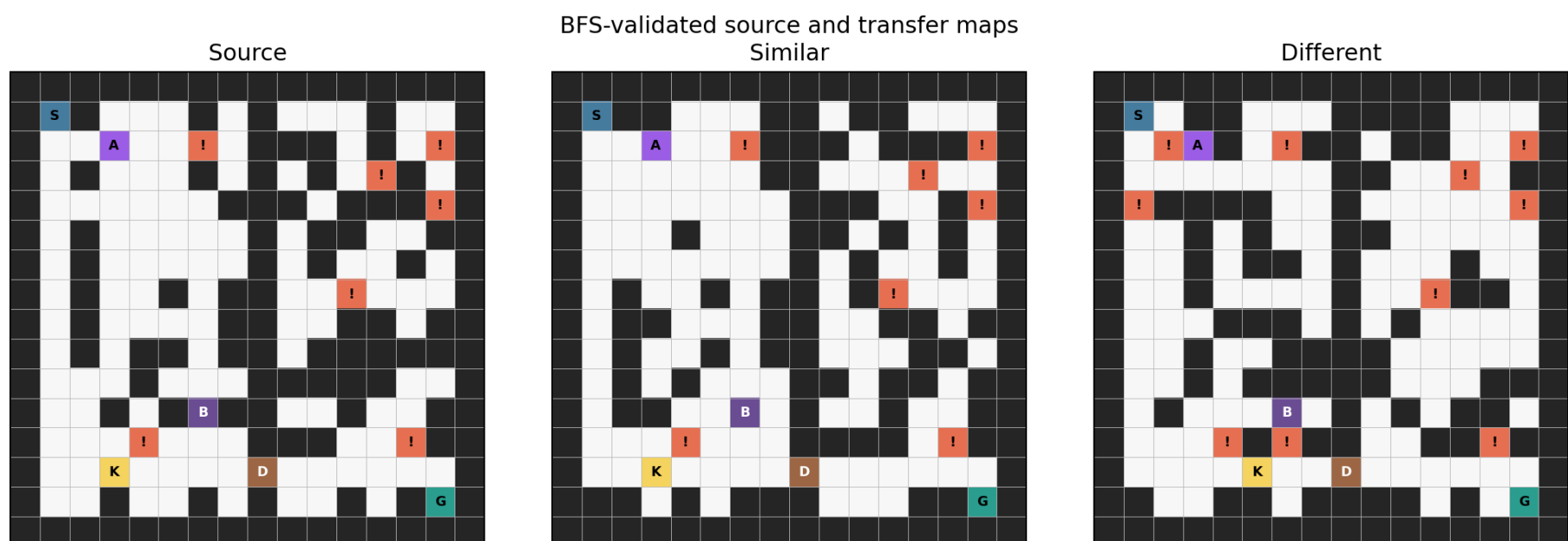
طبق صورت پروژه، رقم یکی مانده به آخر شماره دانشجویی به عنوان Seed پایه در نظر گرفته می شود. در شماره ۴۰۴۰۰۳۵۶ این رقم ۵ است. با قرار دادن این مقدار در رابطه $N = 15 + (b \bmod 4)$ ، اندازه نقشه برابر با $N = 16$ به دست آمد. نقشه مبدأ با همین Seed تولید و سپس در قالب JSON ذخیره شد تا هر سه الگوریتم دقیقاً روی یک ساختار یکسان اجرا شوند.

$$b = \text{int}(\text{student_id}[-2]) = 5, \quad N = 15 + (b \bmod 4) = 16$$

نقشه مبدأ در مجموع ۲۵۶ خانه دارد؛ ۱۲۹ خانه دیوار و ۱۲۷ خانه قابل عبور هستند. بنابراین دیوارها ۵۰.۳۹ درصد نقشه را تشکیل می دهند که از حداقل ۱۵ درصد تعیین شده در صورت پروژه بیشتر است. همچنین هفت خانه جریمه در نقشه قرار دارد. مختصات شروع (۱,۱)، کلید (۱۳,۳)، در (۱۳,۸) و هدف (۱۴,۱۴) است. در ستون ۸ یک سد ساختاری ایجاد شده و تنها مسیر عبور آن از در می گذرد. در نتیجه، دریافت کلید صرفاً یک مرحله نمایشی نیست و بدون آن مأموریت کامل نمی شود.

جدول 1 - اجزای ثابت نقشه مبدأ

نقش	موقعیت یا تعداد	نماد	عنصر
حالت اولیه بدون کلید	(۱,۱)	S	شروع
تغییر has_key از ۰ به ۱	(۱۳,۳)	K	کلید
قبل از کلید غیرقابل عبور	(۱۳,۸)	D	در
پایان موفق فقط همراه کلید	(۱۴,۱۴)	G	هدف
انتقال دوطرفه در یک گام	(۲,۳) و (۱۱,۶)	A/B	انتقال دهنده
افزایش هزینه ورود	۷ خانه	X	خانه جریمه
باقی ماندن در حالت و دریافت جریمه	۱۲۹ خانه	#	دیوار



شکل ۱ - نقشه مبدأ، مقصد مشابه و مقصد متفاوت که همگی با BFS اعتبارسنجی شده اند

شکل ۱ نمایی کلی از هر سه نقشه ارائه می دهد. در مقصد مشابه، محل شروع، کلید، در، هدف، انتقال دهنده ها و خانه های جریمه ثابت مانده و فقط تعدادی از موانع جابه جا شده اند. در مقصد متفاوت، کلید از (۱۳,۳) به (۱۳,۵) منتقل شده، سه خانه جریمه جدید افزوده شده و تغییرات دیوارها گسترده تر است. با وجود این تفاوت ها، سد ساختاری و امکان تکمیل مأموریت در هر دو مقصد حفظ شده است. تعداد دیوارها و خانه های قابل عبور نیز در هر سه نقشه یکسان نگه داشته شد تا ابعاد فضای جدولی تغییر نکند.

۲-۲- اعتبارسنجی مسیر با BFS

برای اعتبارسنجی نقشه، یک BFS صرفاً مکانی کافی نبود؛ زیرا رسیدن به هدف بدون کلید نباید یک مسیر معتبر به شمار می‌آمد. به همین دلیل، جست‌وجو روی حالت افزوده (row, column, has_key) انجام شد و اثر در و انتقال‌دهنده نیز در جابه‌جایی‌ها لحاظ شد. کوتاه‌ترین مسیر مأموریت در نقشه مبدأ ۲۰ عمل است و در مقصد مشابه نیز ۲۰ عمل طول دارد، در حالی که طول مسیر مقصد متفاوت ۱۶ عمل است. البته کوتاه‌تر بودن این مسیر لزوماً به معنای ساده‌تر بودن محیط نیست، زیرا مقصد متفاوت جریمه‌های بیشتری دارد و ساختار دیوارهای آن نیز تغییر کرده است.

BFS در این پروژه فقط برای اطمینان از وجود مسیر و محاسبه فاصله پتانسیل استفاده شد و در هیچ مرحله‌ای جای عامل یادگیرنده را نگرفت. سیاست‌های گزارش‌شده مستقیماً از Value Iteration، Q-Learning یا SARSA به دست آمده‌اند. اطلاعاتی مانند تعداد و نسبت دیوارها، تعداد خانه‌های جریمه، طول مسیر مأموریت و شدت تغییر نقشه مقصد در فایل map_validation.json ذخیره شده است.

۲-۳- انتقال‌دهنده دوطرفه

برای افزودن پویایی بیشتر به محیط، یک جفت انتقال‌دهنده دوطرفه با نمادهای A و B در نظر گرفته شد. وقتی عامل وارد یکی از این دو خانه می‌شود، در همان گام به خانه مقابل منتقل خواهد شد. این رفتار هم در transition_distribution و هم در تابع step پیاده‌سازی شده و با رویداد teleport در لاگ ثبت می‌شود. در مسیر نمونه عامل، ورود به A باعث انتقال از بخش بالایی نقشه به B در بخش پایین می‌شود و راه رسیدن به کلید را کوتاه‌تر می‌کند.

ثابت بودن محل A و B باعث می‌شود برای حفظ خاصیت مارکوف نیازی به متغیر حالت جدیدی نباشد؛ زیرا نتیجه ورود به انتقال‌دهنده فقط به حالت فعلی، عمل اجراشده و نقشه بستگی دارد. اگر انتقال‌دهنده یک‌بارمصرف یا وابسته به زمان بود، لازم بود وضعیت فعال یا غیرفعال بودن آن نیز به حالت اضافه شود. در پیاده‌سازی فعلی، چنین اطلاعات پنهانی یا وابستگی به تاریخچه وجود ندارد.

۳- مدلسازی فرایند تصمیم‌گیری مارکوف

۳-۱- اجزای فرایند تصمیم‌گیری مارکوف

محیط در قالب $M=(S,A,P,R,\gamma)$ مدل‌سازی شد. حالت عامل از سطر، ستون و وضعیت دریافت کلید تشکیل می‌شود. فضای عمل شامل چهار جهت است و اعمال در کد به ترتیب $up=0$ ، $down=1$ ، $left=2$ و $right=3$ شماره‌گذاری شده‌اند. تعریف احتمال انتقال، پاداش، حالت پایانی و سیاست در هر سه الگوریتم بر همین ساختار مشترک استوار است.

$$s = (r, c, k), \quad k \in \{0,1\}, \quad A = \{up, down, left, right\}$$

جدول 2 - تعریف اجزای MDP

شاهد پیاده‌سازی	تعریف در پروژه	جزء
environments/maze.py	(row,column,has_key) روی خانه‌های قابل عبور	فضای حالت S
ACTION_NAMES و ACTION_DELTAS	چهار جهت بالا، پایین، چپ و راست	فضای عمل A
transition_distribution	۰.۸ عمل منتخب و ۰.۱ برای هر انحراف عمود	تابع انتقال P
RewardSpec و potential	potential-shaped و نسخه sparse نسخه	تابع پاداش R
full.json	مقدار اصلی ۰.۹۵؛ تحلیل VI با ۰.۸۰، ۰.۹۵، ۰.۹۹	ضریب کاهش γ
is_terminal	هدف همراه با has_key=1	حالت پایانی
agents/*	حریصانه نسبت به V یا Q؛ ϵ -greedy در آموزش	سیاست

۳-۲- اندازه و دسترسی‌پذیری فضای حالت

از نظر اسمی، برای هر یک از ۱۲۷ خانه قابل عبور دو وضعیت مربوط به کلید وجود دارد؛ بنابراین ۲۵۴ حالت معتبر مکانی-کلیدی خواهیم داشت. برای ساده‌تر شدن اندیس‌گذاری، آرایه‌های جدولی با ابعاد $16 \times 16 \times 2$ و جدول Q با ابعاد $4 \times 16 \times 16 \times 2$ تعریف شدند. در نتیجه، خانه‌های دیوار و برخی حالت‌های دسترس‌ناپذیر نیز در آرایه حضور دارند، اما هنگام استخراج و مقایسه سیاست نادیده گرفته می‌شوند. جست‌وجوی دسترسی‌پذیری از نقطه شروع، ۱۶۸ حالت غیرپایانی را شناسایی کرد و مقایسه دقیق سیاست‌های sparse و shaped نیز روی همین ۱۶۸ حالت انجام شد.

درج متغیر has_key در تعریف حالت برای حفظ خاصیت مارکوف ضروری است. حضور عامل در یک مختصات مشخص پیش و پس از دریافت کلید، در مواجهه با در نتیجه یکسانی ندارد. وقتی k به حالت افزوده می‌شود، پیامد هر عمل فقط از روی حالت و عمل فعلی قابل تعیین است و دیگر نیازی به دانستن تاریخچه مسیر نیست. پس از برداشتن کلید نیز مقدار این متغیر در تمام انتقال‌های بعدی برابر با ۱ باقی می‌ماند.

۳-۳- مدل انتقال تصادفی

برای هر عمل انتخاب‌شده، سه عمل اجرایی محتمل وجود دارد. اگر عامل جهت بالا یا پایین را انتخاب کند، ممکن است به چپ یا راست منحرف شود؛ در انتخاب چپ یا راست نیز انحراف‌های ممکن بالا و پایین هستند. احتمال اجرای عمل اصلی ۰.۸ و احتمال هر انحراف ۰.۱ است. Value Iteration امید ریاضی را روی هر سه پیامد محاسبه می‌کند، در حالی که Q-Learning و SARSA در هر گام یکی از این پیامدها را با مولد تصادفی نمونه‌گیری می‌کنند.

$$P(a_{\text{exec}}=a \mid a_{\text{selected}}=a)=0.8, \quad P(a_{\text{exec}}=a_{\text{perp1}})=P(a_{\text{exec}}=a_{\text{perp2}})=0.1$$

برخورد با مرز نقشه یا دیوار باعث می‌شود عامل در همان موقعیت باقی بماند. تلاش برای عبور از در بدون کلید نیز یک self-loop ایجاد می‌کند، هرچند نوع رویداد و مقدار جریمه آن متفاوت است. هنگام ورود به A یا B، ابتدا حرکت معتبر تشخیص داده می‌شود و سپس عامل به انتقال‌دهنده جفت منتقل می‌شود. هدف فقط در حالتی پایانی است که $k=1$ باشد. در مدل انتقال، حالت پایانی جذب‌کننده است و در اجرای اپیزود نیز پس از terminated یا truncated باید محیط reset شود.

۳-۴- موفقیت در پایان و محدودیت گام

از آنجا که نقشه ۱۲۷ خانه قابل عبور دارد، حداکثر طول هر اپیزود سه برابر این تعداد، یعنی ۳۸۱ گام، تعیین شد. رسیدن به هدف همراه با کلید، پرچم terminated را فعال می‌کند؛ اما پایان اپیزود به دلیل رسیدن به سقف گام با truncated مشخص می‌شود. تمایز این دو حالت اهمیت دارد، زیرا فقط پایان واقعی مأموریت مقدار bootstrap را در به‌روزرسانی Q صفر می‌کند و پایان زمانی نباید موفقیت تلقی شود. به همین دلیل terminated، truncated و success به‌صورت جداگانه در فایل‌های اپیزود ثبت شده‌اند.

۴- طراحی تابع پاداش و ثبت رویدادها

۴-۱- پاداش Sparse

در پاداش sparse، هر گام یک هزینه پایه برابر با ۱- دارد. برخورد با دیوار ۴ واحد و ورود به خانه خطر ۱۰ واحد جریمه اضافی ایجاد می‌کند. دریافت کلید ۳۰ واحد و رسیدن به هدف ۱۰۰ واحد پاداش اضافه دارد. تلاش برای عبور از در بسته نیز ۷ واحد جریمه اضافی در پی خواهد داشت. برای عبور از در پاداش تکرارشونده‌ای در نظر گرفته نشده و door_passed برابر صفر است؛ در نتیجه، رفت‌و برگشت روی در نمی‌تواند به راهی برای جمع‌آوری پاداش تبدیل شود.

جدول 3 - مولفه‌های تابع پاداش

منطق انتخاب	پاداش پایه کل	مولفه تنظیمات	رویداد
کاهش حرکت‌های اضافی	-۱	step=-۱	حرکت عادی
جلوگیری از تکرار برخورد	-۵	wall=-۴	برخورد با دیوار
ترجیح مسیر کم‌خطر در صورت امکان	-۱۱	penalty=-۱۰	ورود به جریمه
تقویت پیشرفت اجباری مأموریت	+۲۹	key=+۳۰	دریافت کلید
تنبیه تلاش بدون کلید	-۸	closed door=-۷	در بسته
reward farming حذف امکان	-۱	door=۰	عبور از در
پاداش اصلی تکمیل مأموریت	+۹۹	goal=+۱۰۰	رسیدن به هدف

۴-۲- پاداش پتانسیل مبنا

در نسخه shaped، پاداش sparse همچنان حفظ شده و تنها یک جمله پتانسیل به آن اضافه می‌شود. پتانسیل هر حالت برابر با منفی فاصله قطعی باقی‌مانده تا تکمیل مأموریت است که با BFS روی حالت شامل وضعیت کلید محاسبه می‌شود. پتانسیل حالت هدف صفر در نظر گرفته شد. ضریب shaping برابر با ۰.۲۵ است و مقدار γ در جمله shaping با γ الگوریتم یکسان نگه داشته شده؛ این برابری نیز در کد کنترل می‌شود.

$$R'(s,a,s') = R(s,a,s') + 0.25[\gamma\Phi(s') - \Phi(s)], \quad \Phi(s) = -d_{mission}(s)$$

این روش به حرکت‌هایی که فاصله باقی‌مانده مأموریت را کم می‌کنند بازخورد می‌دهد، بدون آنکه ساختار potential-based پاداش از بین برود. بررسی Value Iteration نشان داد سیاست‌های بهینه sparse و shaped در هر ۱۶۸ حالت مقایسه‌شده، ۱۰۰ درصد یکسان‌اند. بنابراین shaping در این پروژه سیاست بهینه را تغییر نداده است. مقدار shaping_scale نیز طوری انتخاب شد که حرکت معمولی همچنان هزینه داشته باشد و عامل از تکرار یک حلقه بی‌هدف سود نبرد.

۴-۳- رویدادهای اجباری و نمونه واقعی

تمام رویدادهای موردنیاز در فایل JSONL ثبت شدند. رویدادهای رایج از اپیزودهای طبیعی ارزیابی استخراج شدند و برای دو رویداد کم‌تکرار، دو probe واقعی با استفاده از step محیط اجرا شد. فایل sampled_event_log.jsonl در مجموع ۲۷۳ رکورد از ۱۰ اپیزود طبیعی و ۲ probe دارد. فایل required_event_witnesses.jsonl نیز برای هر رویداد، یک شاهد برگرفته از مدل انتقال یا اجرای واقعی ذخیره می‌کند.

جدول 4 - نمونه پاداش shaped برای رویدادهای اجباری

نتیجه	پاداش ثبت شده	حالت و عمل	رویداد
حرکت به (۲,۱,۰)	-۰.۵۱۲۵	down, (۱,۱,۰)	حرکت عادی
باقی ماندن در همان حالت	-۴.۷۵۰۰	up, (۱,۱,۰)	برخورد دیوار
ورود به (۲,۶,۰)	-۱۱.۰۰۰۰	right, (۲,۵,۰)	ورود جریمه
has_key=۱	+۲۹.۴۰۰۰	down, (۱۲,۳,۰)	دریافت کلید
self-loop	-۷.۸۰۰۰	right, (۱۳,۷,۰)	تلاش در بسته
ورود به در با کلید	-۰.۶۶۲۵	right, (۱۳,۷,۱)	عبور از در
terminated=True	+۹۹.۲۵۰۰	down, (۱۳,۱۴,۱)	رسیدن هدف
truncated=True	-۴.۷۵۰۰	max_steps=۱	سقف گام

۴-۴- بررسی رفتار ناخواسته

در طراحی پاداش سه رفتار ناخواسته به طور مشخص بررسی شد: حرکت حلقه‌ای، جمع‌آوری پاداش فرعی بدون تکمیل مأموریت و دوری بیش از حد از خانه‌های جریمه. کلید فقط یکبار دریافت می‌شود، عبور از در پاداش جداگانه ندارد و رسیدن به هدف اپیزود را پایان می‌دهد. حتی پس از shaping، حرکت عادی همچنان هزینه‌بر یا دست‌کم غیرسودآور باقی می‌ماند. مسیرهای نمونه Q-Learning و SARSA مأموریت را کامل کردند و در لاگ‌ها نیز تکرار عبور از در با هدف کسب پاداش مشاهده نشد. توافق ۱۰۰ درصدی سیاست‌های بهینه sparse و shaped نیز تأیید می‌کند که شکل‌دهی، مسیر بهینه را عوض نکرده است.

۵- ساختار پیاده‌سازی و پروتکل آزمایش

۵-۱- ساختار ماژول‌ها

ساختار کد به شش بخش اصلی محیط، عامل‌ها، انتقال، رابط گرافیکی، آزمایش و تست تقسیم شده است. فایل `environments/maze.py` مدل MDP و فرایند نمونه‌گیری را پیاده می‌کند و `generator.py` مسئول تولید تکرارپذیر نقشه و اجرای BFS است. هر الگوریتم نیز در فایل جداگانه‌ای قرار دارد. `run_experiments.py` ماتریس کامل اجراها را مدیریت می‌کند، در حالی که `analysis.py` فقط با استفاده از داده‌های ذخیره‌شده شکل‌ها را می‌سازد. این جداسازی اجازه می‌دهد نمودارها بدون نیاز به آموزش دوباره بازتولید شوند.

۵-۲- تنظیمات مشترک

جدول 5 - تنظیمات پروفایل `full_experiment`

کاربرد	مقدار	پارامتر
۱۰ اجرای مستقل	۵,۱۷,۲۹,۴۱,۵۳,۶۵,۷۷,۸۹,۱۰۱,۱۱۳	Seed ها
همه آموزش‌های اصلی و shaping	۰.۹۵	γ اصلی
شرط همگرایی max-norm	۱۰-۱e	VI threshold
linear/exponential ϵ	۳۰۰۰ episode, $\alpha=0.15$	Q-Learning
$\lambda=0.3, 0.7, 0.9$	۳۰۰۰ episode, $\alpha=0.12$	SARSA(λ)
۶ مقداردهی اولیه در دو مقصد	۱۵۰۰ episode	Transfer
کاهش خطی یا نمایی	۱.۰ $\rightarrow$ ۰.۰۲ در ۲۴۰۰ اپیزود	ϵ آموزش مبدأ
سازگاری با مقصد	۰.۴ $\rightarrow$ ۰.۰۲ در ۱۲۰۰ اپیزود	ϵ انتقال
سیاست حریصانه و Q ثابت	۲۰۰ episode	ارزیابی نهایی
پیش از آموزش مقصد	۱۰۰ episode	ارزیابی اولیه انتقال
معیار سرعت یادگیری	۰.۸ در پنجره ۱۰۰ اپیزودی	آستانه موفقیت

۵-۳- طراحی تصادفی‌سازی و ارزیابی

برای آنکه نتایج به RNG سراسری وابسته نباشند، Seed هر اپیزود به صورت قطعی و با تابع `episode_seed` از Seed اجرای اصلی، شماره اپیزود و شماره stream ساخته می‌شود. Q-Learning و SARSA در Seed های متناظر از یک stream مشترک برای نويز محیط استفاده می‌کنند، اما مولد تصادفی انتخاب عمل در آن‌ها مستقل است. ارزیابی الگوریتم‌های مبدأ با base seed مشترک ۹۰۰۵ و ارزیابی انتقال با ۲۰۰۰۵ انجام شد؛ بنابراین همه سیاست‌ها با دنباله یکسانی از نويز محیط سنجیده شدند.

در مرحله ارزیابی، ϵ دقیقاً صفر است و عمل حریصانه با یک روش ثابت برای شکستن تساوی انتخاب می‌شود. جدول Q نیز پیش و پس از ارزیابی مقایسه می‌شود تا مشخص شود هیچ به‌روزرسانی ناخواسته‌ای رخ نداده است. برای هر اجرا، نرخ موفقیت، میانگین و انحراف معیار return، میانگین تعداد گام، برخورد با دیوار و ورود به خانه جرمه ذخیره شد. نوارهای خطا در شکل‌ها mean $\pm$ standard deviation بین Seed ها را نشان می‌دهند و نباید با فاصله اطمینان ۹۵ درصد اشتباه گرفته شوند.

۵-۴- اندازه ماتریس آزمایش

جدول 6 - تعداد اجراها و خروجی‌های نهایی

توضیح	ردیف اپیزود	تعداد اجرا	بخش
مرجع shaped؛ حساسیت ۷ جداگانه	۰	۱ summary	مرجع VI
sparse و ۱۰ shaped ۲۰	۹۰۰۰۰	۳۰	Q-Learning
Seed ۱۰ × ۴ λ	۱۲۰۰۰۰	۴۰	SARSA(λ)
تمام اجراها حفظ شده‌اند	۲۱۰۰۰۰	۷۱	جمع مبدأ
۲ مقصد × ۶ سناریو × ۱۰ Seed	۱۸۰۰۰۰	۱۲۰	انتقال
تمام آرایه‌ها finite	۴۰۴ array	۱۹۶ NPZ	مدل‌ها
تولیدشده از داده خام	-	۱۴ PNG	شکل‌ها

این ترکیب از اجراها مانع آن می‌شود که نتیجه فقط براساس بهترین Seed گزارش شود. فایل training_episodes.csv همه ۲۱۰۰۰۰ اپیزود محیط مبدأ و transfer_episodes.csv همه ۱۸۰۰۰۰ اپیزود محیط مقصد را نگه می‌دارد. خلاصه هر اجرا، از جمله اجراهای ناموفق، در run_summary.csv و transfer_summary.csv ثبت شده است. هش کد و فایل پیکربندی نیز در manifest قرار دارد.

۶- الگوریتم Value Iteration

۶-۱- پیاده‌سازی و به‌روزرسانی بلمن

الگوریتم Value Iteration به‌صورت جدولی و بدون استفاده از پیاده‌سازی آماده نوشته شد. در هر تکرار، ابتدا نسخه‌ای از V قبلی نگه داشته می‌شود و سپس مقدار همه ۱۶۸ حالت غیرپایانی و قابل‌دسترسی به‌طور هم‌زمان به‌روزرسانی می‌گردد. برای هر عمل، امید ریاضی پاداش و ارزش ادامه با توجه به سه خروجی احتمالی ۰.۱/۰.۱/۰.۸ محاسبه می‌شود. اگر خروجی به حالت terminal برسد، جمله مربوط به ادامه مسیر صفر در نظر گرفته خواهد شد. پس از همگرایی نیز Q مدل برای تمام زوج‌های حالت-عمل محاسبه و عمل دارای بیشترین مقدار به‌عنوان سیاست حریصانه ذخیره می‌شود.

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_k(s')]$$

$$\Delta_k = \max_s |V_{k+1}(s) - V_k(s)|, \quad \text{stop if } \Delta_k < 10^{-10}$$

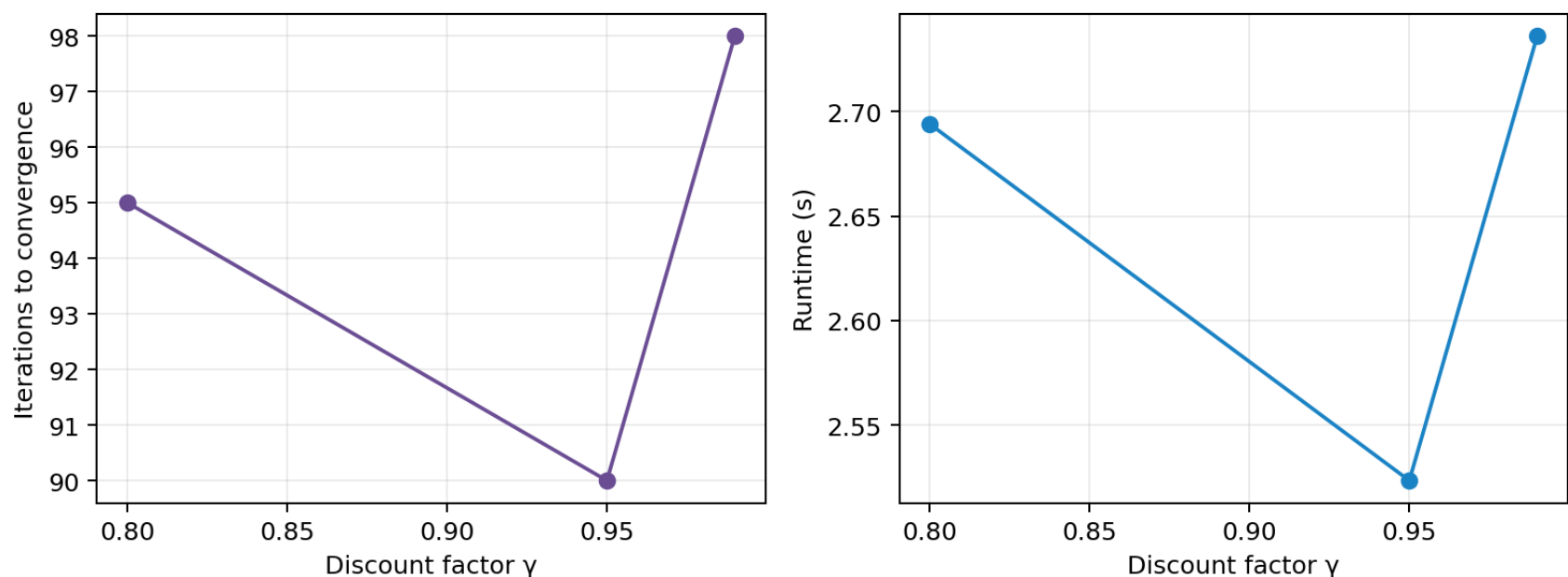
برای اطمینان مستقل از همگرایی، Bellman residual پس از پایان تکرارها یک‌بار دیگر مستقیماً از روی مدل انتقال محاسبه شد. مقادیر V و Q ، سیاست نهایی، تاریخچه Δ ، تعداد backupها، زمان اجرا، γ و آستانه همگرایی همگی در فایل NPZ ذخیره شدند. در هیچ مرحله‌ای از تابع آماده Value Iteration در کتابخانه‌های دیگر استفاده نشده است.

۶-۲- حساسیت به ضریب کاهش

جدول 7 - نتایج Value Iteration با پاداش sparse

γ	تکرار	Bellman backup	زمان (s)	Residual
۰.۸۰	۹۵	۶۳۸۴۰	۲.۶۹۴	۶.۷۲۰×10^{-11}
۰.۹۵	۹۰	۶۰۴۸۰	۲.۵۲۴	۴.۶۴۳×10^{-11}
۰.۹۹	۹۸	۶۵۸۵۶	۲.۷۳۶	۵.۷۳۸×10^{-11}

Value Iteration sensitivity to γ (same sparse reward)



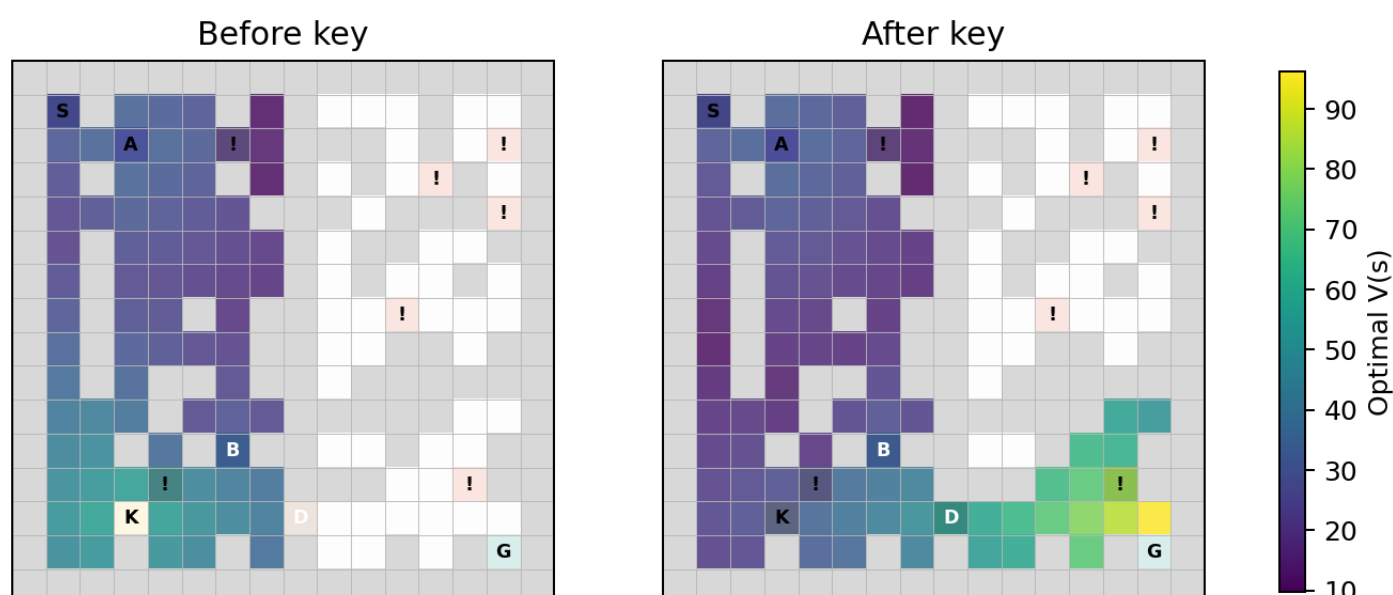
شکل ۲ - تعداد تکرار و زمان همگرایی Value Iteration برای سه مقدار γ

نتایج شکل ۲ نشان می‌دهد رابطه تعداد تکرارها با γ در این نمونه کاملاً یکنواخت نیست. مقدار $\gamma=0.95$ پس از ۹۰ تکرار همگرا شد، در حالی که $\gamma=0.80$ و $\gamma=0.99$ به‌ترتیب به ۹۵ و ۹۸ تکرار نیاز داشتند. با تکیه بر این سه نقطه نمی‌توان نتیجه‌ای کلی درباره کمتر بودن زمان همگرایی در $\gamma=0.95$ مطرح کرد؛ زیرا ساختار پاداش، مسیرهای انتقال‌دهنده و آستانه سخت‌گیرانه نیز بر تعداد تکرارها اثر می‌گذارند. آنچه با اطمینان می‌توان گفت این است که هر سه اجرا به آستانه تعیین‌شده رسیده‌اند و residual آن‌ها در مرتبه 10^{-11} قرار دارد.

زمان‌های جدول به یک اجرای مشخص روی سخت‌افزار فعلی مربوط‌اند و نباید به‌عنوان معیار مطلق پیچیدگی در نظر گرفته شوند. در مقایسه اصلی سه الگوریتم، نسخه shaped با $\gamma=0.95$ به‌عنوان مرجع استفاده شد؛ این اجرا ۴.۸۱۶ ثانیه زمان برد و شامل ۶۰۴۸۰ Bellman backup بود. بیشتر شدن زمان نسبت به نسخه sparse عمدتاً از محاسبه پتانسیل و هزینه انتقال در پاداش shaped ناشی می‌شود، نه از افزایش تعداد تکرارها.

۶-۳- نقشه حرارتی و سیاست بهینه

Value Iteration heatmap - shaped reward

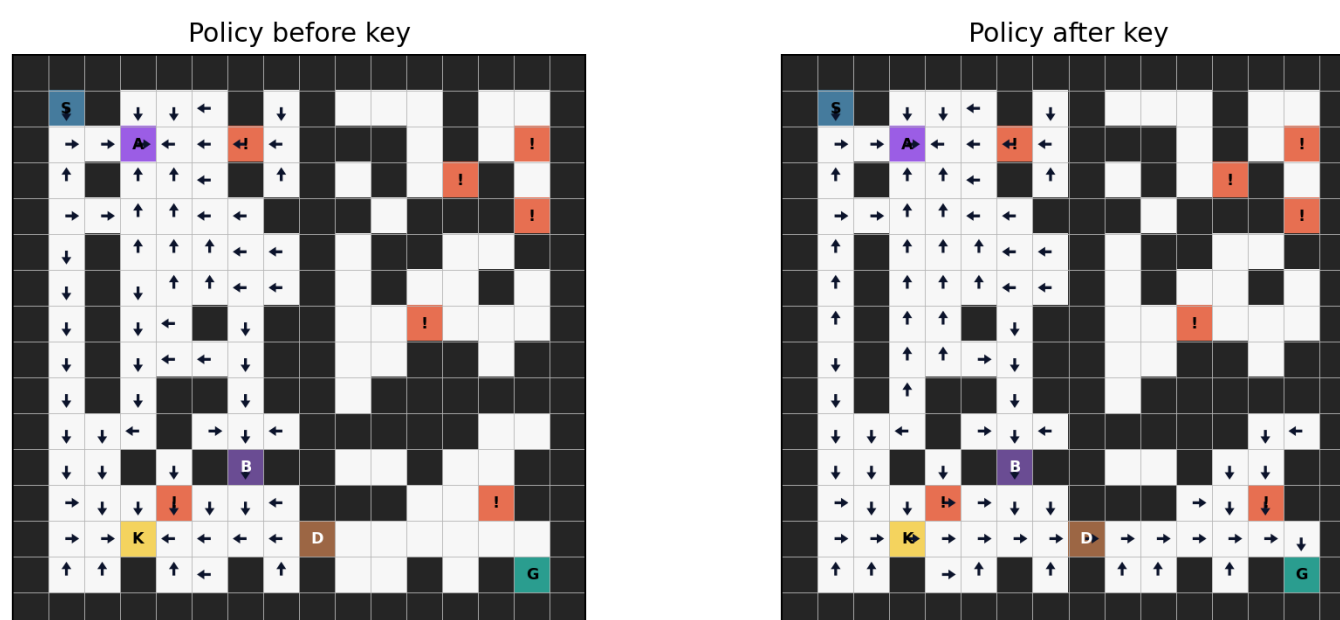


شکل ۳ - تابع ارزش بهینه *shaped* پیش و پس از دریافت کلید

در شکل ۳، تابع ارزش پیش از دریافت کلید و پس از آن در دو لایه جدا نمایش داده شده است. در لایه نخست، مسیرهایی که عامل را به پورتال و سپس به قسمت پایین نقشه می‌رسانند ارزش بیشتری دارند و در اطراف کلید نیز افزایش ارزش دیده می‌شود. پس از برداشتن کلید، بیشترین مقادیر در راهروی پایین و مسیر میان در و هدف قرار می‌گیرند. تفاوت روشن این دو لایه نشان می‌دهد *has_key* صرفاً یک متغیر کمکی در برنامه نیست و مستقیماً ساختار تابع ارزش را تغییر می‌دهد.

خانه‌های دیوار و حالت‌های دسترس‌ناپذیر در شکل ماسک شده‌اند. مقدار ارزش در خود حالت هدف صفر ذخیره می‌شود، زیرا پاداش رسیدن به هدف در انتقال منتهی به حالت *terminal* دریافت شده است. بنابراین صفر بودن *V* در خانه هدف به این معنا نیست که رسیدن به آن ارزشی ندارد.

Optimal policy with terminal and semantic tiles



شکل ۴ - سیاست بهینه Value Iteration همراه با فلش عمل و نشانه عناصر محیط

شکل ۴ عمل حریصانه هر حالت معتبر را با فلش نشان می‌دهد. پیش از دریافت کلید، فلش‌های بخش بالایی عامل را به سمت A هدایت می‌کنند تا از طریق B وارد بخش پایین نقشه شود و به کلید برسد. پس از دریافت کلید، سیاست در امتداد راهروی ردیف ۱۳ از در می‌گذرد و مسیر هدف را دنبال می‌کند. در اطراف خانه‌های جریمه، جهت انتخابی فقط تابع کوتاه‌ترین فاصله هندسی نیست و احتمال لغزش به دو جهت عمود نیز در Q مدل اثر می‌گذارد.

۷- الگوریتم Q-Learning

۷-۱- به روزرسانی off-policy

آموزش Q-Learning با جدول Q اولیه صفر و سیاست رفتاری ϵ -greedy انجام شد. هدف TD براساس بیشترین مقدار Q در حالت بعد ساخته می‌شود، حتی اگر عامل در ادامه به دلیل اکتشاف عمل دیگری را اجرا کند؛ به همین دلیل این روش off-policy است. هنگام پایان واقعی مأموریت، مقدار bootstrap صفر می‌شود. در مقابل، اگر اپیزود صرفاً به سقف زمانی برسد، انتقال آخر همچنان مانند یک نمونه غیر terminal پردازش می‌شود تا truncation با موفقیت اشتباه گرفته نشود.

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

در آزمایش اصلی، $\alpha=0.15$ ، $\gamma=0.95$ و تعداد اپیزودها ۳۰۰۰ در نظر گرفته شد. هنگام آموزش، اگر چند عمل حریصانه مقدار برابری داشته باشند، انتخاب بین آن‌ها تصادفی است؛ در غیر این صورت، صفر بودن اولیه جدول می‌توانست باعث ترجیح دائمی عمل با index صفر شود. در مرحله ارزیابی، $\epsilon=0$ است و تساوی‌ها به صورت ثابت شکسته می‌شوند تا نتیجه هر بار قابل بازتولید باشد.

۷-۲- برنامه‌های کاهش Epsilon

دو برنامه کاهش خطی و نمایی برای ϵ بررسی شدند. در هر دو برنامه، ϵ از ۱.۰ آغاز می‌شود و طی ۲۴۰۰ اپیزود به ۰.۰۲ می‌رسد. برنامه خطی در هر اپیزود مقدار ثابتی از ϵ کم می‌کند، اما برنامه نمایی آن را با یک نسبت ثابت کاهش می‌دهد و در بخش ابتدایی آموزش زودتر به مقادیر کوچک می‌رسد. پس از اپیزود ۲۴۰۰، مقدار ϵ در هر دو برنامه روی ۰.۰۲ ثابت می‌ماند.

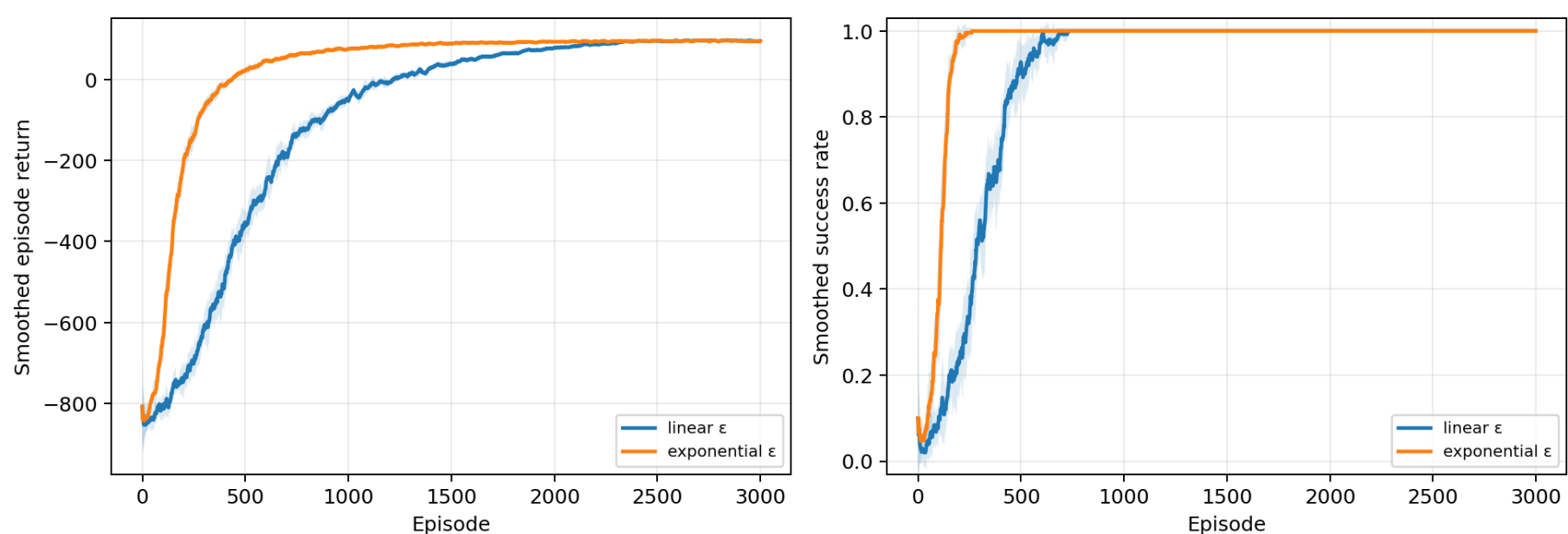
$$\epsilon_{\text{linear}}(t) = \epsilon_0 + \min(t/T, 1)(\epsilon_f - \epsilon_0)$$

$$\epsilon_{\text{exp}}(t) = \epsilon_0 (\epsilon_f / \epsilon_0)^{\min(t/T, 1)}$$

جدول ۸ - مقایسه دو برنامه ϵ در Q-Learning shaped

توافق سیاست (%)	زمان (s)	نمونه کل	اپیزود تا آستانه	برنامه
81.01 ± 2.10	9.835 ± 0.192	۱۷۰۴۵۵.۹	190.1 ± 7.45	نمایی
85.06 ± 1.56	16.162 ± 0.473	۳۱۵۷۰۰.۲	463.0 ± 9.65	خطی

Q-Learning epsilon-schedule sensitivity



شکل ۵ - منحنی return و نرخ موفقیت Q-Learning برای کاهش خطی و نمایی ϵ

براساس شکل ۵، برنامه نمایی خیلی زودتر از برنامه خطی از محدوده returnهای منفی عبور می‌کند و میانگین هموارشده موفقیت آن حدود اپیزود ۲۰۰ به سطح بالایی می‌رسد. در برنامه خطی، اکتشاف شدید مدت بیشتری ادامه دارد و عامل نزدیک اپیزود ۵۰۰ به آستانه مشابهی می‌رسد. این روند با میانگین ۱۹۰.۱ اپیزود برای برنامه نمایی در برابر ۴۶۳ اپیزود برای برنامه خطی تا رسیدن به آستانه ۰.۸ همخوان است.

با این حال، سرعت بیشتر برنامه نمایی با یک مصالحه همراه است. توافق نهایی سیاست آن با مرجع ۸۱.۰۱ درصد است، در حالی که این مقدار برای برنامه خطی به ۸۵.۰۶ درصد می‌رسد. بازده ارزیابی خطی و نمایی به‌ترتیب ۹۳.۷۹۸ و ۹۳.۵۵۶ و میانگین گام آن‌ها ۲۹.۶۰۵ و ۲۹.۸۱۲ است. هر دو روش نرخ موفقیت ۱.۰ دارند، اما برنامه خطی حالت‌های بیشتری را پوشش داده و در کل فضای حالت تطابق بهتری با سیاست مرجع دارد. بر همین اساس، Q-Learning خطی برای مقایسه اصلی و استفاده در GUI انتخاب شد.

۷-۳- بازسازی یک Q-update واقعی

یکی از خواسته‌های صورت پروژ، بازسازی دستی حداقل یک به‌روزرسانی ثبت‌شده در لاگ بود. نخستین رکورد q_update_trace.csv به اپیزود صفر، گام اول و حالت (۱,۱,۰) مربوط است. عامل عمل up با index صفر را انتخاب کرده و با دیوار برخورد کرده است. در این انتقال، پاداش shaped برابر با $r=-4.75$ ثبت شد و Q قبلی و بیشترین Q حالت بعد نیز هر دو صفر بودند.

جدول 9 - اجزای به‌روزرسانی واقعی Q

محاسبه	مقدار	کمیت
برخورد با دیوار و self-loop	$(1,1,0), up, (1,1,0)$	s,a,s'
full.json از	۰.۱۵, ۰.۹۵	α, γ
جدول اولیه صفر	۰.۰۰۰۰	Q_old
پاداش shaped ثبت‌شده	-۴.۷۵۰۰	r
همه اعمال حالت بعد صفر	۰.۰۰۰۰	$\max Q(s',a')$
$r + ۰.۹۵ \times ۰$	-۴.۷۵۰۰	Target
Target-Q_old	-۴.۷۵۰۰	δ
$۰ + ۰.۱۵ \times (-۴.۷۵)$	-۰.۷۱۲۵	Q_new

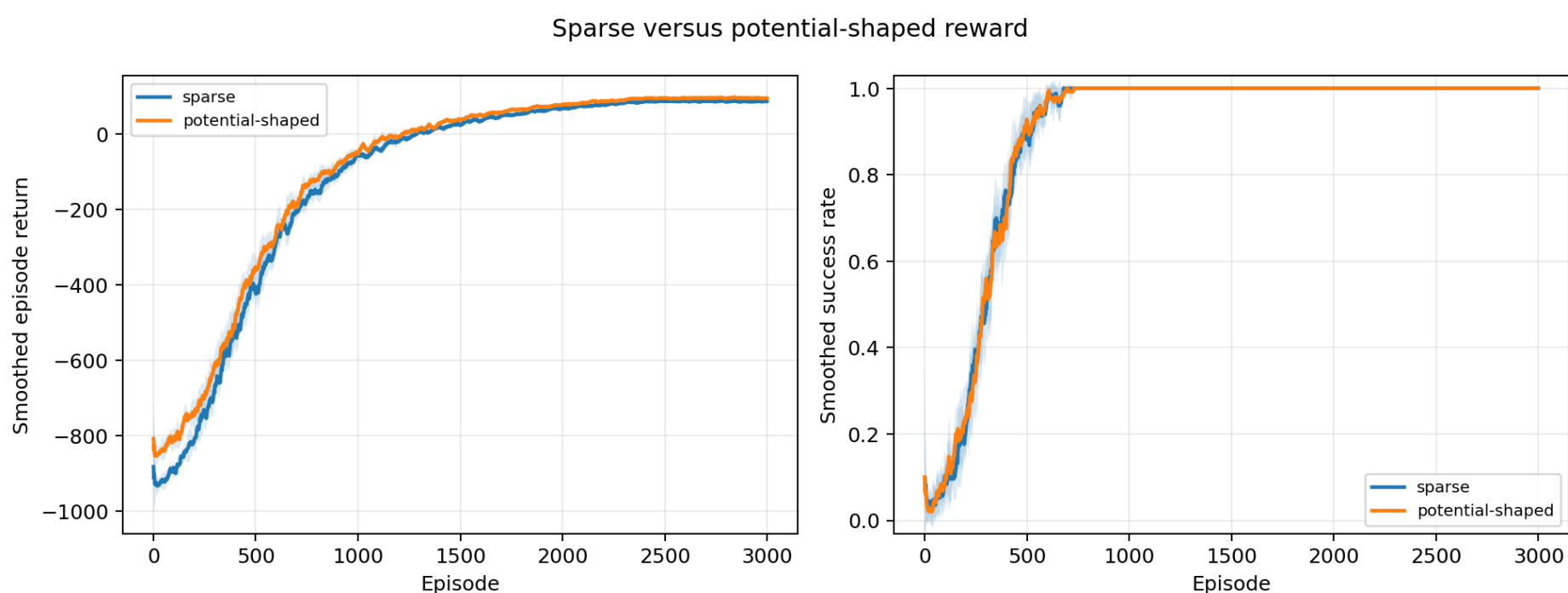
$$Q_{new} = 0 + 0.15[-4.75 + 0.95(0) - 0] = -0.7125$$

رکورد گام پنجم همان اپیزود نشان می‌دهد محاسبه فقط برای مقدارهای اولیه صفر بررسی نشده است. در حالت (۲,۲,۰) و برای عمل up، مقدار قبلی Q برابر با -۰.۷۱۶۲۵، پاداش -۴.۷۷۵ و bootstrap صفر بود. در نتیجه $\delta=-4.05875$ و مقدار جدید Q برابر با -۱.۳۲۵۰۶۲۵ به دست آمد. یکسان بودن این محاسبه دستی با مقدار ثبت‌شده در CSV، یکی از کنترل‌های عددی پیاده‌سازی محسوب می‌شود.

۷-۴- مقایسه پاداش های Shaped و Sparse

جدول 10 - اثر پاداش بر Q-Learning با برنامه خطی

موفقیت	توافق سیاست	بازده ارزیابی	اپیزود تا آستانه	نسخه پاداش
۱.۰	$84.11 \pm 3.06\%$	۸۵.۰۸۸	465.4 ± 11.97	sparse
۱.۰	$85.06 \pm 1.56\%$	۹۳.۷۹۸	463.0 ± 9.65	shaped

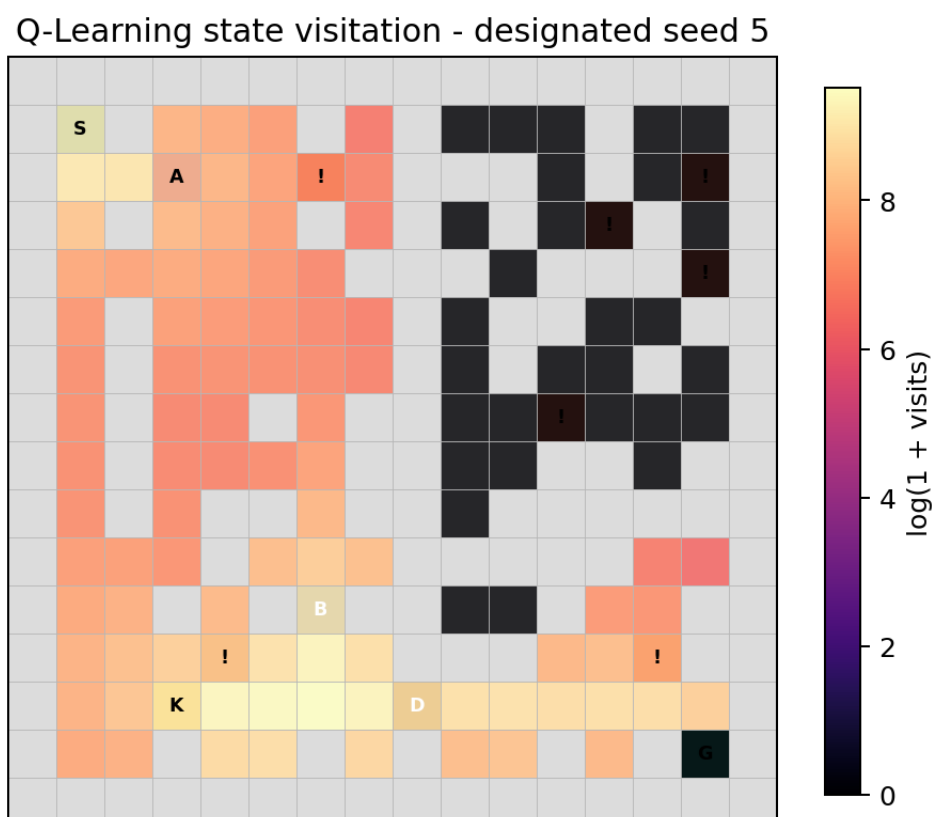


شکل ۶ - مقایسه روند آموزش *Q-Learning* با پاداش *sparse* و *potential-shaped*

همان‌طور که در شکل ۶ دیده می‌شود، نسخه *shaped* در ابتدای آموزش *return* بهتری دارد، اما منحنی موفقیت دو نسخه تقریباً روی یکدیگر قرار می‌گیرد. اختلاف میانگین زمان رسیدن به آستانه فقط ۲.۴ اپیزود است که در مقایسه با پراکندگی میان *Seed*ها مقدار بزرگی نیست. بنابراین نمی‌توان نتیجه گرفت که *shaping* در این نقشه سرعت آموزش را به‌طور چشمگیری افزایش داده است. برداشت محتاطانه‌تر این است که *shaping* بازخورد میانی منظم‌تری فراهم کرده و توافق سیاست را حدود ۰.۹۵ واحد درصد بهبود داده، بی‌آنکه سیاست بهینه تغییر کند.

بیشتر بودن *return* ارزیابی در نسخه *shaped* لزوماً نشانه عملکرد بهتر نیست، زیرا مقیاس پاداش در دو نسخه تفاوت دارد. برای مقایسه منصفانه، نرخ موفقیت، تعداد گام و میزان توافق سیاست معیارهای مناسب‌تری هستند. همچنین در هیچ‌یک از اجراها نشانه‌ای از حلقه پاداش یا توقف عمدی پیش از رسیدن به هدف مشاهده نشد.

۷-۵- بازدید حالت و مسیر نهایی



شکل ۷ - نقشه لگاریتمی تعداد بازدید حالت‌ها در *Q-Learning* خطی با *Seed=۵*

شکل ۷ نشان می‌دهد بیشترین تعداد بازدید در اطراف نقطه شروع، ورودی و خروجی پورتال و راهروی میان کلید و در رخ داده است. تبدیل $\log(1 + \text{visits})$ برای آن به کار رفت که اختلاف‌های بزرگ تعداد بازدید نیز در شکل قابل مشاهده باشند. خانه‌های بخش بالای سمت راست، چون برای تکمیل مأموریت ضروری نیستند، نمونه‌های بسیار کمتری دریافت کرده‌اند. همین توزیع نامتوازن توضیح می‌دهد که چرا نرخ موفقیت ۱۰۰ درصد الزاماً به توافق کامل سیاست در همه حالت‌های قابل دسترسی منجر نمی‌شود.



شکل ۸ - کوتاه‌ترین مسیر موفق ثبت‌شده در ارزیابی حریصانه *Q-Learning*

مسیر نمونه نمایش‌داده‌شده در شکل ۸ از ۲۰ عمل تشکیل شده است. عامل ابتدا از نقطه شروع به A می‌رسد و در همان گام به B منتقل می‌شود؛ سپس از بخش پایین نقشه به کلید می‌رود، از در عبور می‌کند و به هدف می‌رسد. این مسیر از میان ۲۰۰ اپیزود ارزیابی تصادفی به‌عنوان یک نمونه موفق انتخاب شده و نباید آن را با میانگین طول اپیزودها یکی دانست. میانگین *Q-Learning* خطی ۲۹.۶۰۴۵ گام است، زیرا انحراف‌های ۰.۱/۰.۱ در بسیاری از اپیزودها حرکت اضافه یا برخورد با دیوار ایجاد می‌کنند.

۸- الگوریتم SARSA (λ)

۸-۱- بهروزرسانی on-policy و Eligibility Traces

در SARSA، هدف TD از $Q(s', a')$ استفاده می‌کند؛ یعنی همان عملی که واقعاً در حالت بعد و با سیاست رفتاری ϵ -greedy انتخاب شده است. از این رو، هزینه اکتشاف نیز در تخمین مقادیر وارد می‌شود و الگوریتم ماهیت on-policy دارد. ردپای شایستگی نیز باعث می‌شود خطای TD علاوه بر زوج حالت-عمل جاری، بر تصمیم‌های فعال قبلی اثر بگذارد.

$$\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$$

$$Q \leftarrow Q + \alpha \delta_t E, \quad E \leftarrow \gamma \lambda E$$

در اجرای نهایی از ردپای replacing استفاده شد. در این حالت، $E(s_t, a_t)$ برابر با ۱ قرار می‌گیرد و بازدید دوباره از یک زوج حالت-عمل مقدار ردپای آن را بیش از ۱ افزایش نمی‌دهد. این انتخاب در محیطی که برخورد با دیوار یا در بسته self-loopهای متعددی ایجاد می‌کند، از رشد بیش از اندازه ردپا جلوگیری می‌کند. حالت accumulating نیز در پیاده‌سازی قابل انتخاب است، اما تمام نتایج این گزارش با replacing به دست آمده‌اند.

۸-۲- تبدیل به SARSA یک مرحله‌ای در $\lambda=0$

وقتی $\lambda=0$ باشد، پس از هر بهروزرسانی همه ردپاها در $\gamma \lambda=0$ ضرب می‌شوند و پیش از گام بعدی از بین می‌روند. بنابراین فقط زوج (s_t, a_t) جاری تغییر می‌کند و الگوریتم دقیقاً به SARSA یک‌مرحله‌ای تبدیل می‌شود. با افزایش λ ، ردپاهای قبلی آهسته‌تر محو می‌شوند و هر خطای جدید δ می‌تواند چند تصمیم گذشته را نیز اصلاح کند. این ویژگی انتشار پاداش را سریع‌تر می‌کند، اما اگر λ بیش از حد بزرگ باشد، نویز و خطای نمونه نیز در فاصله طولانی‌تری منتشر خواهد شد.

۸-۳- بازسازی چند گام واقعی از E و δ

فایل sarsa_trace.csv همه ردپاهای فعال را پیش و پس از decay در قالب JSON ذخیره می‌کند. برای $\lambda=0.7$ و $\gamma=0.95$ ، ضریب decay برابر با ۰.۶۶۵ است. جدول ۱۱ پنج گام نخست اپیزود صفر را نشان می‌دهد. تا گام چهارم، تعداد ردپاهای فعال از ۱ به ۴ افزایش می‌یابد. در گام پنجم، زوج حالت-عمل گام قبل دوباره دیده می‌شود و replacing مقدار ردپای جاری را مجدداً روی ۱ قرار می‌دهد؛ به همین دلیل تعداد ردپاها همچنان ۴ است و مجموع L_1 پیش از decay با ادامه ساده دنباله قبلی تفاوت دارد.

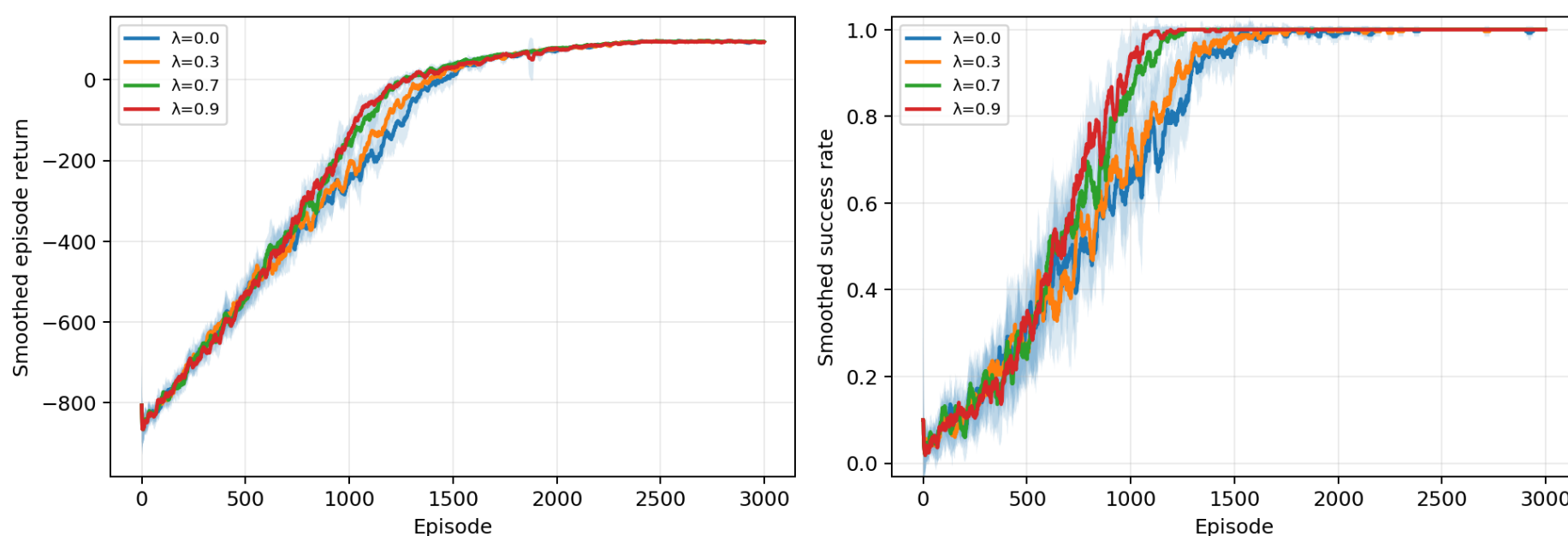
جدول ۱۱ - نمونه واقعی trace در SARSA با $\lambda=0.7$

گام	δ	ردپای فعال	E قبل	E بعد
۱	-۴.۷۵۰۰۰	۱	۱.۰۰۰۰۰۰	۰.۶۶۵۰۰۰
۲	-۰.۵۱۲۵۰	۲	۱.۶۶۵۰۰۰	۱.۱۰۷۲۲۵
۳	-۰.۵۲۵۰۰	۳	۲.۱۰۷۲۲۵	۱.۴۰۱۳۰۵
۴	-۴.۷۷۵۰۰	۴	۲.۴۰۱۳۰۵	۱.۵۹۶۸۶۸
۵	-۴.۷۴۶۳۵	۴	۱.۹۳۱۸۶۸	۱.۲۸۴۶۹۲

در گام دوم، ردپای عمل up در حالت شروع پیش از decay مقدار ۰.۶۶۵ دارد. بنابراین خطای $\delta = -۰.۵۱۲۵$ ، علاوه بر عمل جاری، مقدار Q تصمیم قبلی را نیز به اندازه تقریبی $\alpha \delta E \approx 0.12 \times (-0.5125) \times 0.665 = -0.0409$ تغییر می‌دهد. پس از decay، ردپای قبلی به ۰.۴۴۲۲۲۵ و ردپای جاری به ۰.۶۶۵ می‌رسد. این رکورد عملاً نشان می‌دهد چند trace به‌طور هم‌زمان در بهروزرسانی فعال‌اند و λ فقط به نام الگوریتم اضافه نشده است.

جدول 12 - نتایج SARSA(λ) روی Seed 10

λ	توافق سیاست	SD بین Seed	بازده ارزیابی	اپیزود تا آستانه
۰.۰	$75.65 \pm 1.93\%$	۰.۱۹۳	۹۳.۹۵۷	1196.0 ± 80.1
۰.۳	$77.14 \pm 2.47\%$	۰.۱۰۸	۹۳.۹۳۱	1127.6 ± 101.0
۰.۷	$79.82 \pm 1.63\%$	۰.۱۴۹	۹۳.۸۹۹	959.9 ± 34.9
۰.۹	$79.05 \pm 2.18\%$	۱۴.۳۰۲	۸۸.۲۷۳	916.1 ± 65.1

SARSA(λ) sensitivityشکل ۹ - روند $return$ و موفقیت $SARSA$ برای λ های ۰.۰، ۰.۳، ۰.۷ و ۰.۹

شکل ۹ بیانگر آن است که λ های بزرگ‌تر در میانه آموزش زودتر به نرخ موفقیت بالا می‌رسند. $\lambda=0.9$ با میانگین ۹۱۶.۱ اپیزود سریع‌ترین گزینه است، اما میانگین $return$ آن در ارزیابی نهایی به ۸۸.۲۷۳ کاهش یافته و انحراف معیار میان $Seed$ ها به ۱۴.۳۰۲ رسیده است. میانگین طول اپیزود ۳۱۰.۸۱۵ گام و تعداد برخورد با دیوار ۳.۳۸۸۵ نیز برای این مقدار از سایر λ ها نامطلوب‌تر است. در نتیجه، سریع‌تر رسیدن به آستانه لزوماً به معنای یادگیری سیاستی پایدارتر نیست.

$\lambda=0.3$ کمترین پراکندگی $return$ را دارد، اما برای رسیدن به آستانه به‌طور متوسط حدود ۱۶۸ اپیزود بیشتر از $\lambda=0.7$ نیاز دارد. مقدار $\lambda=0.7$ پس از ۹۵۹.۹ اپیزود به آستانه می‌رسد، $return$ ارزیابی آن ۹۳.۸۹۹ با $SD=0.149$ است و با ۷۹.۸۲ درصد، بیشترین توافق سیاست را در میان چهار مقدار λ به دست می‌آورد. بنابراین در چارچوب این پروژه، $\lambda=0.7$ بهترین موازنه میان سرعت، پایداری و کیفیت سیاست را فراهم می‌کند.

۹- مقایسه سه الگوریتم

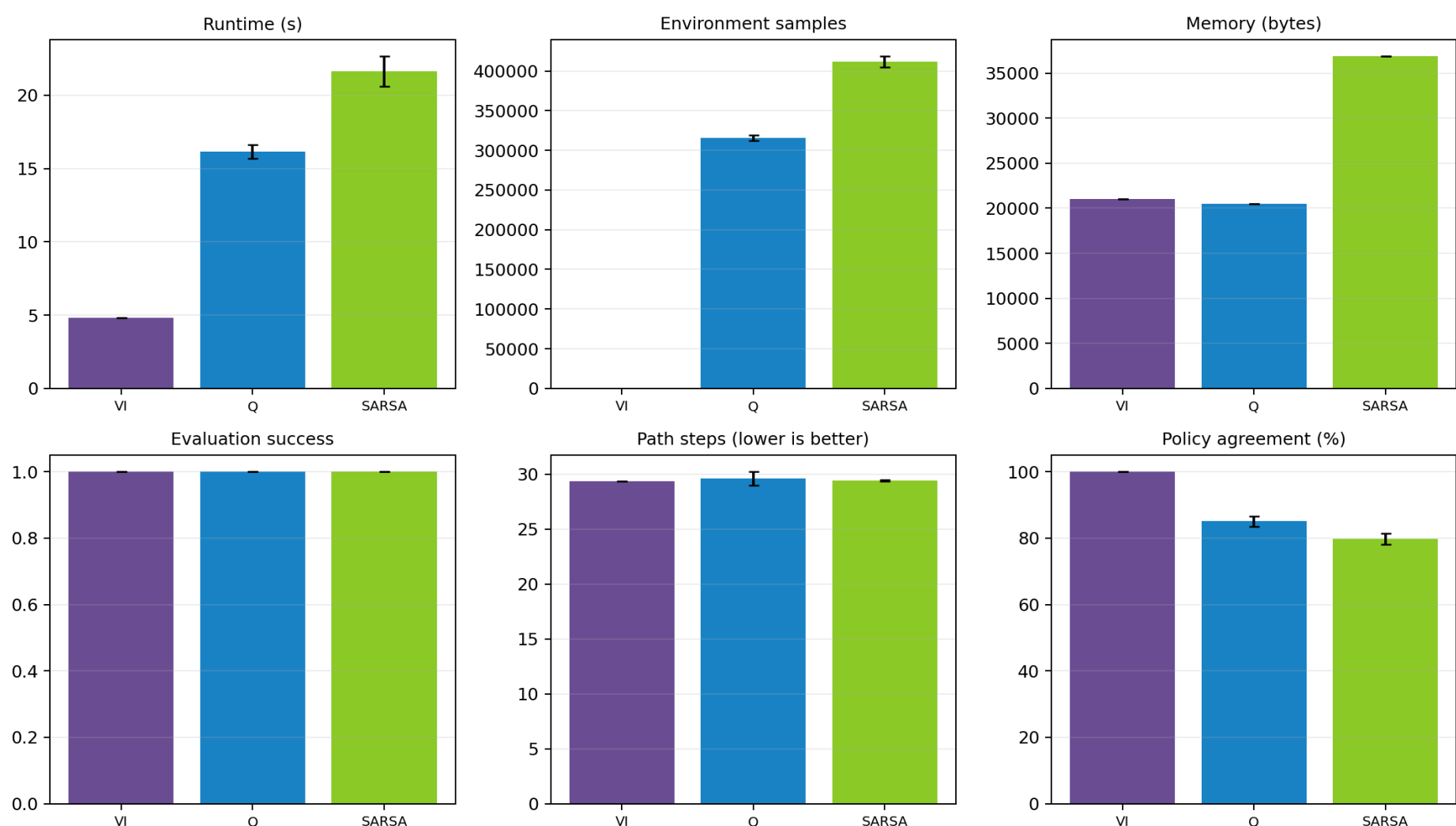
۹-۱- مقایسه عددی در شرایط یکسان

برای مقایسه اصلی، هر سه الگوریتم روی نقشه مبدأ، پاداش shaped و $\gamma=0.95$ ارزیابی شدند. نسخه خطی Q-Learning و SARSA با $\lambda=0.7$ در این مقایسه به کار رفتند. نتایج روش‌های مدل‌آزاد به‌صورت میانگین ۱۰ Seed همراه با انحراف معیار میان Seedها گزارش شده‌اند. Value Iteration فقط یک اجرای قطعی دارد؛ بنابراین پراکندگی صفر آن را نباید معادل پایداری آماری حاصل از ۱۰ اجرای مستقل دانست.

جدول 13 - مقایسه اصلی سه الگوریتم

معیار	VI	Q-Learning	SARSA(۰.۷)	برداشت
زمان (s)	۴.۸۱۶	16.162 ± 0.473	21.633 ± 1.011	VI سریع‌تر در این اندازه
نمونه محیط	۰	315700 ± 3139	411628 ± 6773	VI مدل کامل مصرف می‌کند
Bellman backup	۶۰۴۸۰	-	-	هزینه محاسبات مدل‌محور
حافظه آرایه (byte)	۲۰۹۹۲	۲۰۴۸۰	۳۶۸۶۴	E حافظه SARSA را زیاد می‌کند
موفقیت ارزیابی	۱.۰	۱.۰	۱.۰	برای رتبه‌بندی کافی نیست
میانگین گام	۲۹.۳۷۰	۲۹.۶۰۵	۲۹.۴۲۲	اختلاف کوچک
بازده ارزیابی	۹۳.۸۶۱	۹۳.۷۹۸	۹۳.۸۹۹	عملاً نزدیک
توافق با VI	۱۰۰٪	۸۵.۰۶٪	۷۹.۸۲٪	اختلاف در حالت‌های کم‌بازدید

Three-algorithm comparison (mean $\pm$ across-seed SD)



شکل ۱۰ - مقایسه زمان، نمونه، حافظه، موفقیت، طول مسیر و توافق سیاست سه الگوریتم

طبق شکل ۱۰، Value Iteration در این نقشه کوچک هم از نظر زمان و هم از نظر طول مسیر اندکی بهتر عمل کرده است، اما این برتری با فرض دسترسی کامل به مدل انتقال به دست می‌آید. صفر بودن تعداد نمونه‌های محیط در VI به معنای نبود هزینه محاسباتی نیست، زیرا ۶۰۴۸۰ backup روی خروجی‌های مختلف انتقال انجام شده است. اگر مدل ناشناخته باشد یا فضای حالت بسیار بزرگ شود، این مزیت لزوماً حفظ نخواهد شد.

Q-Learning در مقایسه با SARSA(0.7) به نمونه و زمان کمتری نیاز داشت و حافظه آرایه‌های آن ۲۰۴۸۰ بایت بود. SARSA علاوه بر Q و visits، در طول هر اپیزود یک آرایه eligibility هم‌اندازه Q نگه می‌دارد؛ از این رو مصرف حافظه ثبت‌شده آن به ۳۶۸۶۴ بایت می‌رسد. در مقابل، SARSA میانگین گام اندکی کمتر و پراکندگی return کمی پایین‌تر دارد. اختلاف ۰.۱۸ گام و چند صدم return بسیار کوچک‌تر از آن است که بتوان براساس آن برتری قطعی یکی از روش‌ها را نتیجه گرفت.

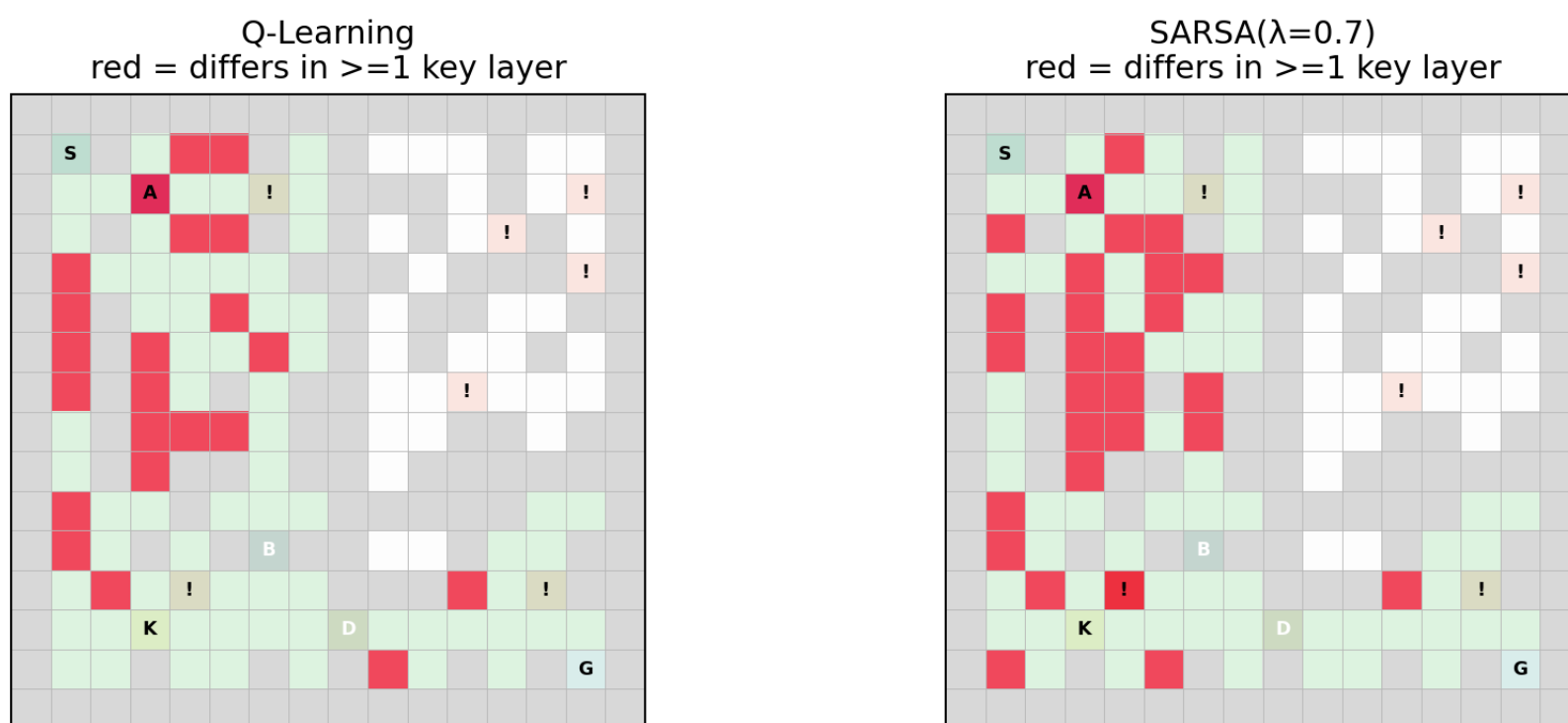
۹-۲- رفتاری on-policy و off-policy در نزدیکی خطر

از دید نظری، Q-Learning هدف خود را با بهترین عمل حالت بعد می‌سازد و هزینه عمل اکتشافی بعدی را در target وارد نمی‌کند. در مقابل، SARSA از عملی استفاده می‌کند که سیاست ε-greedy واقعاً در حالت بعد انتخاب کرده است؛ بنابراین پیامد اکتشاف نیز در مقادیر یادگرفته‌شده آن منعکس می‌شود. با وجود این تفاوت، داده‌های این پروژه فاصله بزرگی از نظر ایمنی سیاست حریم‌نهایی نشان نمی‌دهند. میانگین ورود به خانه جریمه در ارزیابی Q-Learning برابر با ۰.۵۳۸۵ و برای SARSA(0.7) برابر با ۰.۵۵۱۰ است. میانگین برخورد با دیوار نیز به ترتیب ۲.۴۶۴۵ و ۲.۴۴۵۵ ثبت شد.

در ۱۰۰ اپیزود پایانی آموزش نیز میانگین ورود به خانه جریمه برای Q-Learning و SARSA به ترتیب ۰.۴۸۵ و ۰.۴۸۴ و تعداد برخورد با دیوار ۲.۴۷۶ و ۲.۵۰۸ بود؛ یعنی اختلاف‌ها بسیار ناچیزند. احتمالاً کاهش ۴ تا ۰.۰۲، استفاده هر دو الگوریتم از پاداش shaped و وجود یک مسیر غالب برای مأموریت، این شباهت را ایجاد کرده است. در نتیجه، ادعای کلی «SARSA همیشه ایمن‌تر است» با شواهد این پروژه تأیید نمی‌شود. تفاوت دو روش بیشتر در مقادیر Q، سرعت رسیدن به آستانه و میزان توافق سیاست دیده می‌شود تا تعداد نهایی رویدادهای خطر.

۹-۳- اختلاف سیاست با مرجع

Model-free greedy policy disagreement with Value Iteration



شکل ۱۱ - حالت‌های اختلاف سیاست Q-Learning و SARSA(0.7) با Value Iteration

در شکل ۱۱، هر خانه قرمز نشان می‌دهد که سیاست مدل‌آزاد دست‌کم در یکی از دو وضعیت کلید با سیاست VI تفاوت دارد. بیشتر اختلاف‌ها در شاخه‌ها و نواحی کم‌بازدید قرار گرفته‌اند که عبور از آن‌ها برای مسیر اصلی ضروری نیست. توافق میانگین Q-Learning بیشتر است، باین‌حال هر دو روش با وجود اختلاف در بخش‌هایی از سیاست، همه اپیزودهای ارزیابی را با موفقیت به پایان رسانده‌اند. این مشاهده تفاوت دو معیار را روشن می‌کند: نرخ موفقیت فقط رفتار عامل از نقطه شروع را می‌سنجد، اما policy agreement تصمیم‌های عامل را در تمام حالت‌های قابل‌دسترسی بررسی می‌کند.

جدول 14 - سه نمونه اختلاف سیاست Q-Learning با VI در Seed=5

تفسیر محلی	Regret دقیق	عمل VI	عمل Q	حالت
راهروی عمودی؛ بازدید محدود جهت پایین	۵.۴۰۳	down	up	(۶,۱,۰)
شاخه میانی پیش از کلید	۵.۱۷۶	down	up	(۷,۳,۰)
حالت بعد از کلید خارج از مسیر غالب	۴.۳۴۶	up	down	(۸,۳,۱)

برای نمونه، در حالت (۶,۱,۰) مقدار Q مدل برای down برابر با ۲۵.۰۰۲ و برای up برابر با ۱۹.۵۹۹ است، در حالی که جدول یادگرفته‌شده up را ۱۷.۵۱۱ و down را فقط ۶.۶۴۸ برآورد کرده است. وضعیت مشابهی در حالت (۷,۳,۰) دیده می‌شود. این خطاها ناشی از یک مانع مستقیم نیستند و بیشتر به توزیع نامتوازن بازدیدها، تصادفی بودن نمونه‌ها و محدود بودن آموزش به ۳۰۰۰ اپیزود مربوط می‌شوند. حالت سوم نیز پس از دریافت کلید قرار دارد و کمتر در مسیر غالب دیده شده است. مقادیر regret از مدل دقیق مقصد محاسبه شدند، نه از اختلاف خام Qهای یادگرفته‌شده.

برای SARSA هم سه نمونه جداگانه ذخیره شد. در یکی از آن‌ها، یعنی حالت (۹,۳,۰)، SARSA عمل up را انتخاب می‌کند، در حالی که VI عمل down را با regret برابر با ۶.۲۸۹ ترجیح می‌دهد. مقادیر Q در SARSA برای این حالت منفی‌ترند، زیرا سیاست on-policy با ϵ نهایی غیرصفر هزینه اکتشاف را نیز تجربه کرده است. این اختلاف‌ها به شکست مأموریت منجر نشدند، اما نشان می‌دهند موفقیت در مسیر شروع تا هدف به‌تنهایی تضمین نمی‌کند که کل جدول Q به سیاست بهینه همگرا شده باشد.

۱۰- انتقال یادگیری روی Q-Learning

۱۰-۱- طراحی دو محیط مقصد

انتقال یادگیری فقط برای Q-Learning با برنامه خطی انجام شد. در هر Seed، جدول Q آموخته شده در نقشه مبدأ ذخیره و سپس برای مقداردهی اولیه نقشه مقصد به کار رفت. هر سه نقشه ابعاد ۱۶×۱۶ و ۱۲۹ دیوار دارند تا شکل جدول Q در همه آن‌ها ثابت بماند. معتبر بودن هر دو نقشه مقصد نیز با BFS روی حالت افزوده بررسی شد.

جدول 15 - مقایسه نقشه‌های مبدأ و مقصد

متفاوت	مشابه	مبدأ	ویژگی
۴۰.۳۱۰٪	۱۷.۸۲۹٪	-	تغییر موانع نسبت به مبدأ
ثابت	ثابت	(۱,۱)	شروع
(۱۳,۵)	ثابت	(۱۳,۳)	کلید
ثابت	ثابت	(۱۳,۸)، (۱۴,۱۴)	در و هدف
۱۰	۷	۷	خانه جریمه
ثابت	ثابت	(۲,۳)/(۱۱,۶)	پورتال A/B
۱۶	۲۰	۲۰	طول مسیر BFS مأموریت

در مقصد مشابه، ۱۷.۸۲۹ درصد موانع نسبت به مبدأ تغییر کرده‌اند و در نتیجه شرط تغییر ۱۵ تا ۲۰ درصدی صورت پروژه رعایت شده است. مقصد متفاوت بیش از ۳۵ درصد تغییر دارد؛ ضمن آنکه محل کلید عوض شده و سه خانه جریمه جدید به نقشه اضافه شده است. ثابت ماندن هدف با صورت مسئله تعارضی ندارد، زیرا جابه‌جایی کلید یا هدف کافی بود و در این طراحی، کلید تغییر کرده است.

۱۰-۲- چهار روش انتقال و شش سناریوی عملی

چهار رویکرد اصلی انتقال عبارت‌اند از آموزش از صفر، انتقال کامل، انتقال ضریب‌دار و انتقال انتخابی. از آنجا که انتقال ضریب‌دار با سه مقدار β اجرا شد، برای هر مقصد شش سناریوی مقداردهی اولیه در دسترس است. در انتقال کامل، کل آرایه Q مبدأ کپی می‌شود. در انتقال ضریب‌دار، Q اولیه مقصد برابر با βQ_{source} است. روش انتخابی نیز فقط خانه‌هایی را منتقل می‌کند که امضای محلی ۳×۳ آن‌ها در دو نقشه کاملاً یکسان باشد.

$$Q_{T^{\{0\}}}(s,a) = \beta Q_S(s,a), \quad \beta \in \{0.25, 0.50, 0.75\}$$

جدول 16 - سناریوهای مقداردهی اولیه مقصد

تعریف	حالت‌های منتقل شده	سناریو
جدول Q صفر؛ خط مبنا	۰	scratch
کپی کامل آرایه مبدأ	۵۱۲ لایه حالت	full
یک‌چهارم Q مبدأ	۵۱۲	scaled $\beta=0.25$
نصف Q مبدأ	۵۱۲	scaled $\beta=0.50$
سه‌چهارم Q مبدأ	۵۱۲	scaled $\beta=0.75$
فقط امضای محلی یکسان	۳۲	مشابه selective
تقریباً معادل scratch	۲	selective متفاوت

۱۰-۳- نتایج مقصد مشابه

جدول 17 - انتقال به مقصد مشابه؛ میانگین Seed 10

Return نهایی / توافق	نمونه تا آستانه	Return اولیه	موفقیت اولیه	سناریو
۹۷.۷۵۷ / ۶۷.۸۹٪	۱۸۰۸۷.۹	-۱۸۰۹.۷۵	۰.۰۰۰	scratch
۹۷.۱۴۸ / ۷۰.۶۲٪	۵۹۳۲.۶	۸۷.۷۴۹	۱.۰۰۰	full
۹۷.۷۸۶ / ۷۳.۵۴٪	۷۰۸۹.۸	۸۷.۷۴۹	۱.۰۰۰	$\beta=0.25$
۹۷.۳۲۱ / ۷۱.۳۷٪	۵۹۹۵.۹	۸۷.۷۴۹	۱.۰۰۰	$\beta=0.50$
۹۶.۹۶۵ / ۷۱.۲۴٪	۵۹۴۸.۵	۸۷.۷۴۹	۱.۰۰۰	$\beta=0.75$
۹۷.۵۵۸ / ۷۲.۳۰٪	۱۷۵۲۳.۰	-۱۰۰۵.۴۱۱	۰.۰۰۰	selective

در مقصد مشابه، انتقال کامل و هر سه مقدار β از همان ارزیابی اولیه نرخ موفقیت ۱.۰ دارند. یکسان بودن موفقیت و return اولیه β ها قابل انتظار است، زیرا ضرب همه Q ها در یک ضریب مثبت ترتیب argmax را تغییر نمی‌دهد. نرخ موفقیت اولیه scratch و selective صفر است. هرچند selective در مقایسه با scratch مقدار return اولیه بهتری دارد، انتقال تنها ۳۲ لایه برای ساختن یک سیاست کامل از نقطه شروع تا هدف کافی نبوده است.

در سناریوهایی که از ابتدا موفق‌اند، معیار اپیزود تا آستانه مقدار ۱۰۰ را نشان می‌دهد؛ دلیل آن این است که success_window برابر با ۱۰۰ است و پیش از کامل شدن این پنجره، معیار تعریف نمی‌شود. معیار تعداد نمونه تفاوت‌ها را بهتر آشکار می‌کند: full با ۵۹۳۳ نمونه به آستانه می‌رسد، در حالی که scratch به ۱۸۰۸۸ نمونه نیاز دارد. پس از ۱۵۰۰ اپیزود، نرخ موفقیت همه سناریوها ۱.۰ است. بیشترین توافق سیاست در مقصد مشابه به $\beta=0.25$ و مقدار ۷۳.۵۴ درصد تعلق دارد؛ از نظر return نهایی نیز scratch و $\beta=0.25$ تقریباً برابرند.

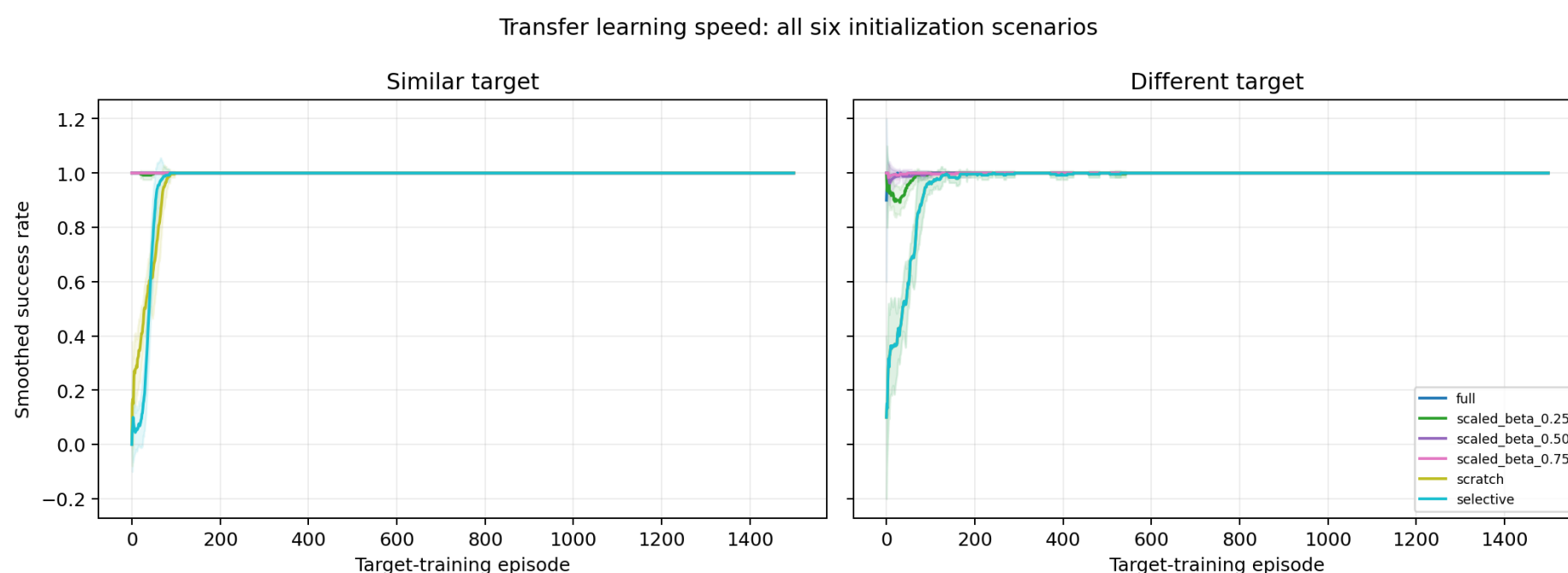
۱۰-۴- نتایج مقصد متفاوت

جدول 18 - انتقال به مقصد متفاوت؛ میانگین Seed 10

Return نهایی / توافق	نمونه تا آستانه	Return اولیه	موفقیت اولیه	سناریو
۸۳.۷۰۹ / ۶۹.۵۹٪	۱۸۲۴۰.۱	-۱۶۷۵.۳۱۹	۰.۰۰۰	scratch
۸۳.۱۰۶ / ۵۷.۷۴٪	۵۹۴۳.۴	-۴۴۱.۶۸۳	0.704 ± 0.014	full
۸۳.۰۱۴ / ۶۶.۲۳٪	۸۱۲۴.۸	-۴۴۱.۶۸۳	0.704 ± 0.014	$\beta=0.25$
۸۳.۱۹۷ / ۶۳.۰۸٪	۶۳۸۶.۳	-۴۴۱.۶۸۳	0.704 ± 0.014	$\beta=0.50$
۸۳.۶۱۵ / ۵۹.۵۲٪	۵۸۵۳.۳	-۴۴۱.۶۸۳	0.704 ± 0.014	$\beta=0.75$
۸۳.۷۰۹ / ۶۹.۵۹٪	۱۸۲۴۰.۱	-۱۶۷۵.۳۱۹	۰.۰۰۰	selective

در مقصد متفاوت، انتقال کامل و انتقال‌های ضریب‌دار از همان ابتدا در حدود ۷۰.۴ درصد اپیزودها موفق هستند؛ بنابراین بخشی از دانش مسیر مبدأ همچنان قابل استفاده است. با این حال، return اولیه منفی باقی می‌ماند و حدود ۲۹.۶ درصد اپیزودها با شکست پایان می‌یابند. scratch و selective در ارزیابی اولیه هیچ موفقیتی ندارند. روش selective تنها دو لایه حالت را منتقل کرده و به همین دلیل رفتار آن عملاً با scratch یکسان است.

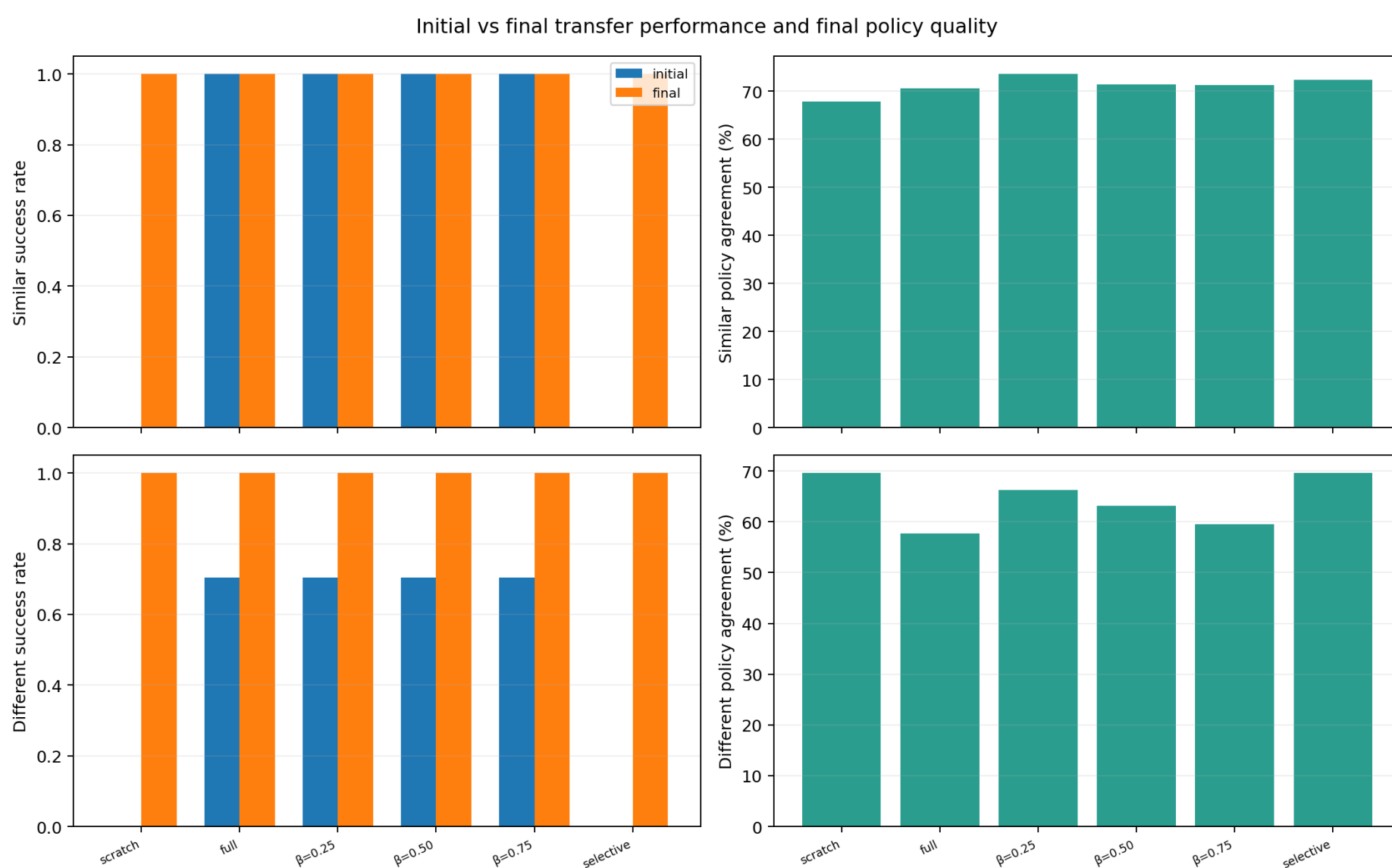
پس از آموزش در مقصد، همه روش‌ها به نرخ موفقیت ۱.۰ می‌رسند، اما کیفیت سیاست نهایی آن‌ها یکسان نیست. توافق سیاست برای scratch و selective برابر با ۶۹.۵۹ درصد است، در حالی که full فقط ۵۷.۷۴ درصد توافق دارد. $\beta=0.25$ بخشی از این افت را جبران کرده و توافق را به ۶۶.۲۳ درصد رسانده است. بنابراین انتقال کامل، با وجود بهبود شروع و sample efficiency، می‌تواند اثر برخی مسیرهای قدیمی را در جدول نهایی حفظ کند. در مقصد متفاوت، بیشترین return نهایی ابتدا برای scratch/selective و سپس برای $\beta=0.75$ به دست آمده است.



شکل ۱۲ - سرعت یادگیری شش سناریوی انتقال در مقصد مشابه و متفاوت

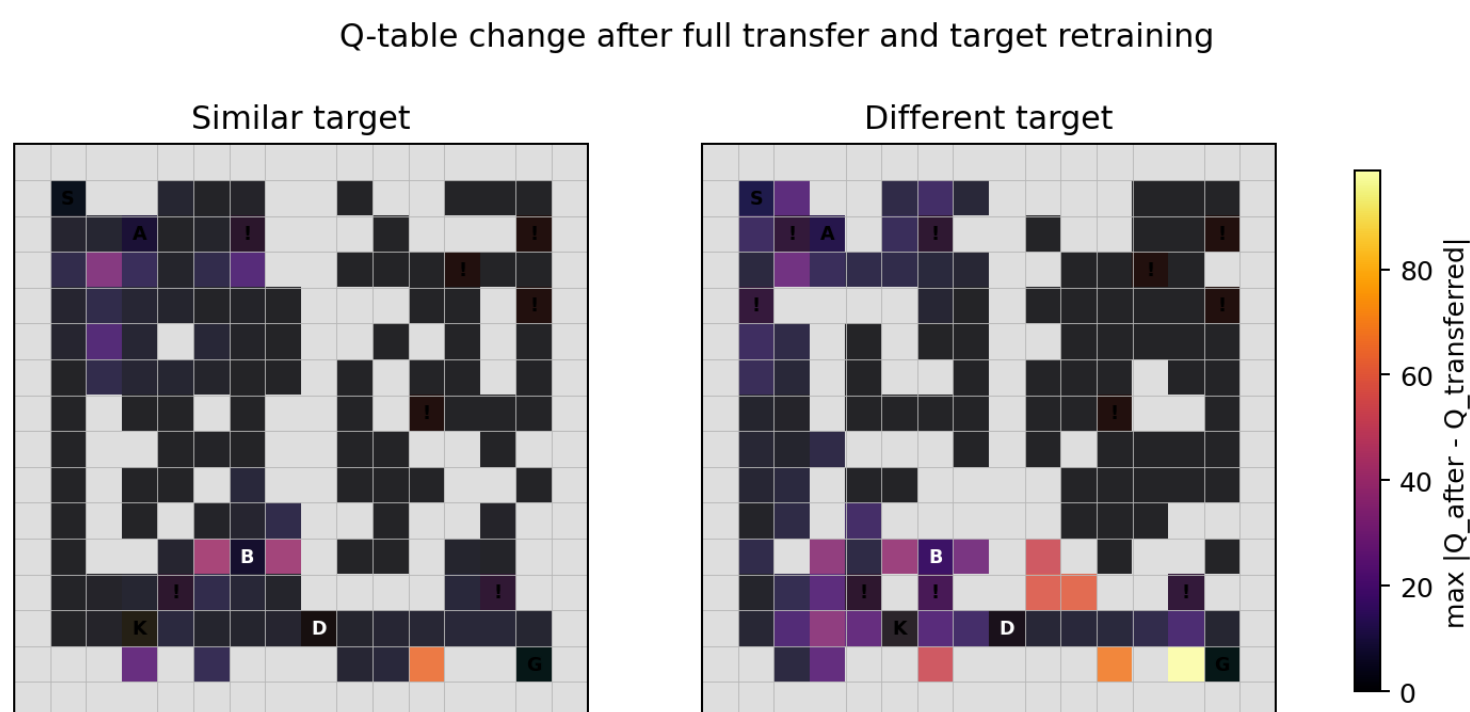
در بخش چپ شکل ۱۲، β و هر سه منحنی از ابتدای آموزش نزدیک نرخ موفقیت ۱.۰ قرار دارند، در حالی که β و β selective و scratch حدود ۱۰۰ اپیزود زمان می‌خواهند تا به سطح مشابهی برسند. در مقصد متفاوت، منحنی‌های انتقال‌یافته ابتدا افت کوتاهی را تجربه می‌کنند و سپس پایدار می‌شوند. این افت نشان می‌دهد حتی دانشی که در آغاز مفید است باید با انتقال‌های جدید محیط سازگار شود. β و scratch از نرخ موفقیت پایین‌تری شروع می‌کنند و با تأخیر بیشتری رشد می‌یابند.

در ارزیابی پیش از آموزش، β بر انتخاب عمل حریصانه اثری ندارد، زیرا ضرب Q ها در یک عدد مثبت argmax را ثابت نگه می‌دارد. اثر واقعی β پس از آغاز یادگیری TD ظاهر می‌شود؛ مقیاس اولیه Q نسبت به پاداش‌های جدید، سرعت جایگزینی ارزش‌های قدیمی و حساسیت به نمونه‌های نخست را تغییر می‌دهد. از آنجا که هیچ β ی در تمام معیارها برتری یکنواختی ندارد، انتخاب آن باید براساس چند شاخص انجام شود و نمی‌توان تنها به یک منحنی موفقیت تکیه کرد.



شکل ۱۳ - موفقیت اولیه و نهایی و توافق سیاست سناریوهای انتقال

شکل ۱۳ تفاوت میان موفقیت و کیفیت سیاست را به خوبی نشان می‌دهد. در پایان آموزش، نرخ موفقیت تمام روش‌ها ۱.۰ است، اما میزان توافق سیاست از حدود ۵۷.۷ تا ۷۳.۵ درصد تغییر می‌کند. اگر فقط final success گزارش شود، تفاوت ماندگار میان انتقال کامل، انتقال ضریب‌دار و scratch دیده نخواهد شد. اثر مثبت انتقال در مقصد مشابه روشن‌تر است؛ در مقصد متفاوت، مزیت شروع سریع‌تر با افت موضعی کیفیت سیاست همراه می‌شود.



شکل ۱۴ - بیشینه تغییر قدر مطلق Q پس از انتقال کامل و آموزش مقصد

بررسی شکل ۱۴ نشان می‌دهد در مقصد مشابه، آموزش مقصد بیشترین اصلاح را در Qهای نواحی تغییرکرده و بخش پایینی مسیر ایجاد کرده است. در مقصد متفاوت، تغییرها هم گسترده‌ترند و هم دامنه بیشتری دارند، به‌خصوص در اطراف کلید جابه‌جاشده و راهروی میان در و هدف. این شکل فقط اختلاف عمل حریمانه را نمایش نمی‌دهد؛ مقدار بیشینه $|Q_{\text{after}} - Q_{\text{transferred}}|$ روی دو لایه کلید و چهار عمل محاسبه شده است. بنابراین حتی در حالت‌هایی که argmax ثابت مانده، ممکن است مقادیر Q تغییر چشمگیری کرده باشند.

۱۰-۶- انتقال منفی و اصلاح رفتار

برای شناسایی انتقال منفی، یک معیار مشخص و قابل محاسبه تعریف شد. ابتدا حالت‌هایی جدا شدند که امضای محلی آن‌ها میان نقشه مبدأ و مقصد تغییر کرده بود. سپس عمل حریصانه Q مبدأ با عمل بهینه مدل مقصد مقایسه شد و regret یک‌مرحله‌ای آن با استفاده از Qهای دقیق Value Iteration مقصد به دست آمد. هر نمونه تنها زمانی به‌عنوان شاهد پذیرفته شد که عمل منتقل‌شده واقعاً زیربهبینه باشد و آموزش مقصد نیز آن را اصلاح کند.

جدول 19 - دو شاهد انتقال منفی در سناریوی full و Seed=5

عمل پس از آموزش	علت تغییر	Regret	عمل بهینه	عمل منتقل‌شده	مقصد/حالت
up	دیوار سمت چپ اضافه شده	۶.۱۷۸	up	left	مشابه (۱۴,۵,۰)
right	کلید از چپ به راست منتقل شده	۱۰.۸۳۵	right	left	متفاوت (۱۳,۴,۰)

در نمونه مقصد مشابه، Q مبدأ برای left مقدار ۴۷.۳۸۵ و برای up مقدار ۴۲.۷۹۳ دارد، اما مدل مقصد up را عمل بهینه می‌داند و regret انتخاب left برابر با ۶.۱۷۸ است. پس از آموزش مقصد، $Q(up)=45.697$ به بیشترین مقدار تبدیل شد. در نمونه مقصد متفاوت، جایگاه قبلی کلید باعث شده Q مبدأ عمل left را ترجیح دهد. پس از انتقال کلید به (۱۳,۵)، عمل right بهینه است و regret عمل قدیمی ۱۰.۸۳۵ محاسبه شد. آموزش مقصد مقدار $Q(right)$ را به ۶۷.۰۲۸ رساند و این تصمیم را اصلاح کرد.

وجود این دو نمونه به معنای شکست کلی انتقال نیست؛ زیرا full در هر دو مقصد همچنان عملکرد اولیه بهتری از scratch دارد. تفسیر دقیق‌تر آن است که انتقال می‌تواند در سطح کلی سودمند باشد، اما در چند حالت خاص تصمیم نامناسبی را نیز وارد مقصد کند. به همین دلیل، ارزیابی انتقال باید عملکرد اولیه، سرعت یادگیری، نتیجه نهایی و خطاهای موضعی را جداگانه گزارش کند.

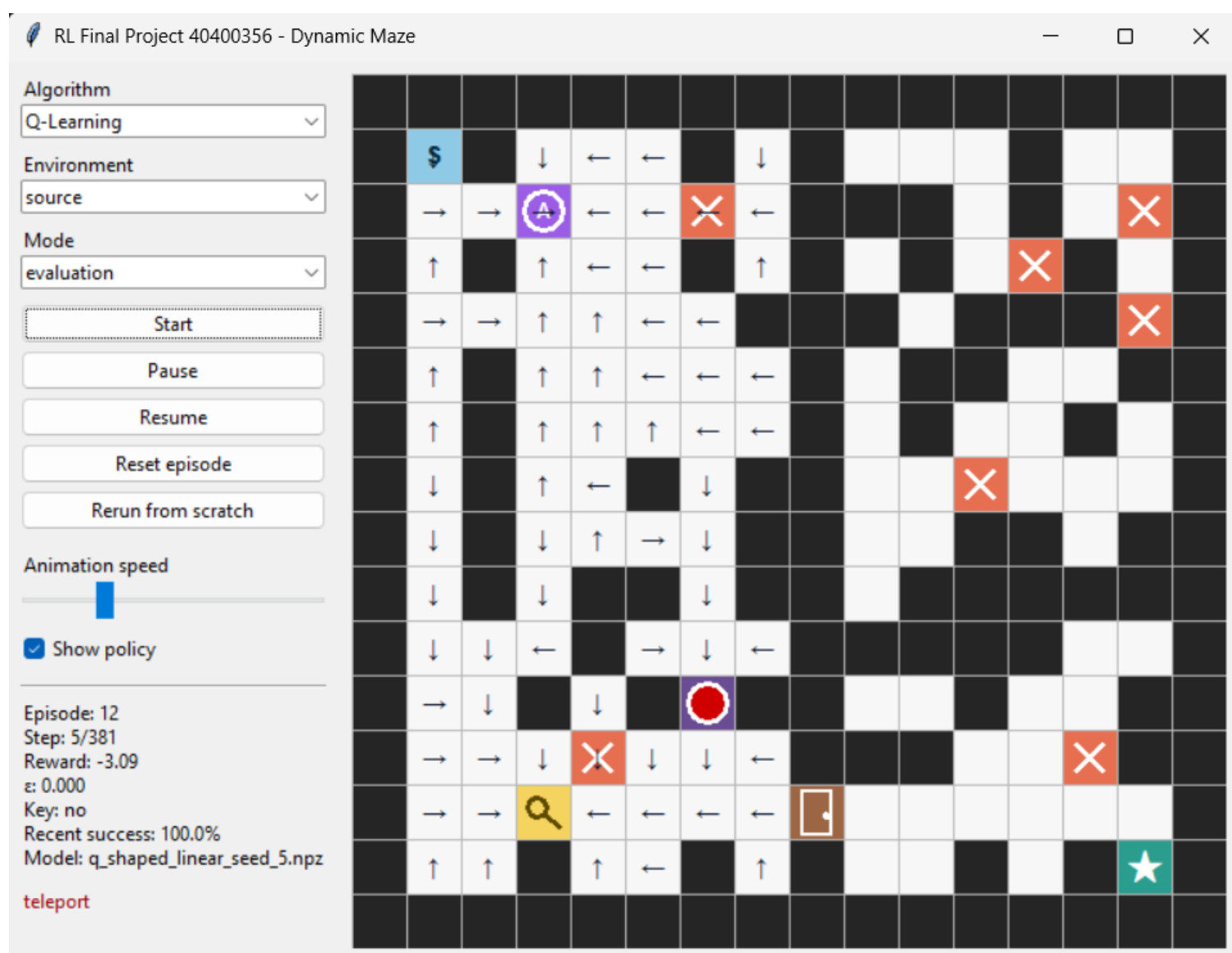
۱۱- رابط گرافیکی و خروجی های بصری

۱۱-۱- قابلیت های رابط

رابط گرافیکی با Tkinter توسعه داده شد و حرکت عامل را گام به گام نمایش می دهد. برای دیوار، خانه عادی، جریمه، کلید، در، هدف، عامل و دو پورتال، رنگ یا نماد جداگانه ای در نظر گرفته شده است. نمایش فلش های سیاست روی نقشه نیز قابل فعال یا غیرفعال کردن است. رویداد جاری، مانند teleport، در پایین پنجره نمایش داده می شود و اطلاعاتی مانند وضعیت کلید، ϵ ، پاداش، شماره اپیزود، گام و نرخ موفقیت اخیر در لحظه به روزرسانی می شوند.

جدول 20 - کنترل ها و اطلاعات GUI

وضعیت آزمون	موارد پیاده سازی شده	گروه
فعال	source/similar/different، محیط training/evaluation	انتخاب
Pause/Resume دستی پاس شد	Start، Pause، Resume، Reset episode، Rerun from scratch	اجرا
فعال	سرعت انیمیشن و Show policy	نمایش
در لحظه به روزرسانی می شود	Episode، Step، Reward، ϵ ، Key، Recent success، Model	اطلاعات
اعتبارسنجی خودکار	بارگذاری checkpoint سازگار با fingerprint نقشه	مدل



شکل ۱۵ - اجرای واقعی GUI در ویندوز در حالت evaluation

در تصویر شکل ۱۵، Q-Learning در محیط source و حالت evaluation در حال اجراست. مدل q_shaped_linear_seed_5.npz بارگذاری شده، $\epsilon=0.000$ و Recent success برابر با ۱۰۰٪ است. عامل در اپیزود ۱۲ و گام ۵ از ۳۸۱ قرار دارد و رویداد teleport نیز ثبت شده است. فلش های سیاست روی خانه های معتبر دیده می شوند. دکمه های Pause و Resume هم در اجرای واقعی ویندوز آزمایش شدند و عملکرد درستی داشتند.

با تغییر حالت از training به evaluation، جدول Q موجود حفظ می‌شود و ϵ روی صفر قرار می‌گیرد. اگر مدل فعالی در حافظه نباشد، رابط یک checkpoint سازگار را بارگذاری می‌کند. metadata هر مدل شامل نوع الگوریتم، نقشه، ψ ، مقدار λ در صورت نیاز و fingerprint معنایی نقشه است. اگر مدل معتبری پیدا نشود، GUI به‌جای نمایش نتیجه غیرواقعی، خطا را به‌صورت شفاف اعلام می‌کند.

۱۱-۲- خروجی‌های تصویری خواسته شده

چهارده شکل ذخیره‌شده همه خروجی‌های تصویری موردنیاز را پوشش می‌دهند: نقشه‌ها، Heatmap ارزش، سیاست VI، حساسیت ψ ، برنامه‌های ϵ ، مقایسه پاداش، میزان بازدید حالت‌ها، مسیر نهایی، حساسیت λ ، مقایسه سه الگوریتم، اختلاف سیاست، منحنی‌های انتقال، عملکرد انتقال و تغییر Q. فایل figure_manifest.json مشخص می‌کند هر شکل از کدام فایل یا مدل ساخته شده است. همه نمودارها مستقیماً از داده‌های JSON، CSV یا NPZ تولید شده‌اند و اعداد آن‌ها به‌صورت دستی تغییر نکرده‌اند.

۱۲- آزمون، صحت و بازتولیدپذیری

۱۲-۱ آزمون‌های اجرا شده

مجموعه آزمون‌ها شامل ۴۵ تست واحد و یکپارچه است. دامنه این تست‌ها به `import` شدن ماژول‌ها یا اجرای بدون خطا محدود نیست؛ معادله دقیق `Q`، پایان `terminal`، فعال بودن چند `trace` در `SARSA`، احتمال‌های `۰.۱/۰.۱/۰.۸`، `fingerprint` نقشه، انتقال منفی، بارگذاری مدل در `GUI` و پاک‌سازی خروجی‌های قدیمی نیز بررسی می‌شوند. اجرای کامل تست‌ها در ویندوز ۳۸.۷۸۹ ثانیه طول کشید و با پیام `Ran ۴۵ tests` و نتیجه `OK` پایان یافت.

جدول 21 - شواهد صحت نسخه نهایی

کنترل	نتیجه	شاهد
تست واحد/یکپارچه	۴۵/۴۵ OK	<code>unittest discover -s tests -v</code>
Verifier	<code>status=verified</code>	<code>verify_project.py</code>
مدل‌ها	۱۹۶ archive / ۴۰۴ finite array	<code>model_validation</code>
شکل‌ها	۱۴ PNG معتبر	<code>figure_manifest</code> و <code>SHA-۲۵۶</code>
اجراهای مبدأ	۷۱	<code>run_summary.csv</code>
اجراهای انتقال	۱۲۰	<code>transfer_summary.csv</code>
Residual بیشینه	<code>۶.۷۲۰e-۱۱</code>	<code>vi_gamma_sensitivity.csv</code>
sparse/shaped سیاست	<code>۱۰۰% agreement</code>	<code>reward_design_verification.json</code>
GUI	<code>evaluation. teleport. Pause/Resume</code> پاس	آزمون ویندوز و تست <code>headless</code>
هش منبع	<code>۰e۱۰۹۴۰d...۶۱۶۸ac۸۸</code>	<code>run_manifest.json</code>

۱۲-۲ کنترل‌های بسته و داده

اسکرپت `verify_project.py` همه آرشیوهای `NPZ` را با `allow_pickle=False` باز می‌کند و وجود `metadata`، شکل صحیح آرایه‌ها و `finite` بودن اعداد را می‌سنجد. همچنین ۱۴ تصویر را `decode` می‌کند، تعداد اجراها و ردیف‌ها را با مقادیر موردانتظار تطبیق می‌دهد و چند مدل منتخب `GUI` را دوباره ارزیابی می‌کند. هش کد مستقل از سیستم‌عامل محاسبه می‌شود و فقط فایل‌های کد، تنظیمات و نقشه را دربر می‌گیرد؛ پوشه `results`، محیط مجازی و `cache` در آن لحاظ نمی‌شوند. پیش از آغاز هر اجرای کامل، خروجی‌های تولیدشده قبلی حذف می‌شوند تا هیچ فایل `stale` در نتایج باقی نماند. مقدار `status=completed` در `manifest` فقط پس از پایان ماتریس آزمایش، اعتبارسنجی مدل‌ها و تولید همه شکل‌ها ثبت می‌شود. در اجرای نهایی، شمارش ۲۱۰۰۰۰ ردیف مبدأ، ۱۸۰۰۰۰ ردیف انتقال، ۱۹۶ مدل و ۱۴ شکل دقیقاً با مقادیر موردانتظار برابر بود.

۱۲-۳ دستورات بازتولید

نصب و اجرای پروژه به مسیر خاصی در رایانه وابسته نیست. پس از ساخت محیط مجازی و نصب `requirements.txt`، رابط گرافیکی با `main.py` و آزمایش کامل با `run_experiments.py` اجرا می‌شود. برای بازسازی شکل‌ها بدون تکرار آموزش نیز می‌توان از گزینه `analysis-only` استفاده کرد. از آنجا که اجرای کامل زمان‌بر است، داده‌های پروفایل `full` از قبل در بسته نهایی قرار داده شده‌اند.

۱۳- پاسخ مستقیم به پرسش‌های تحلیلی صورت پروژه

۱۳-۱- تعریف کامل MDP و حفظ خاصیت مارکوف

در تعریف کامل MDP، فضای حالت به صورت (row, column, has_key) و فضای عمل شامل چهار جهت در نظر گرفته شد. تابع انتقال، نویز ۰.۱/۰.۱/۰.۸ و قواعد مربوط به دیوار، در، کلید، هدف و پورتال را پوشش می‌دهد. تابع پاداش در دو نسخه sparse و shaped تعریف شده، ضریب کاهش اصلی ۰.۹۵ است و تنها رسیدن به هدف همراه با کلید حالت پایانی محسوب می‌شود. سیاست VI از argmax روی Q مدل و سیاست دو روش مدل آزاد از جدول Q آموخته شده استخراج می‌شود. تعریف دقیق این اجزا در فایل‌های maze.py، full.json و source_map.json ثبت شده است.

قرار دادن has_key در حالت، خاصیت مارکوف را حفظ می‌کند؛ زیرا اثر در بر آینده عامل فقط به موقعیت فعلی و مقدار k وابسته است. پورتال‌ها ثابت و دوطرفه‌اند و وضعیت پنهانی ندارند. بنابراین برای تعیین خروجی transition_distribution دانستن s و a کافی است و نیازی به تاریخچه حرکت‌ها نیست. تست same_state_action_has_same_model_independent_of_history نیز همین ویژگی را بررسی می‌کند.

۱۳-۲- تفاوت on-policy و off-policy در رفتار واقعی

Learning مستقل از عمل رفتاری بعدی و با هدف max Q آموزش می‌بیند، در حالی که SARSA عمل بعدی انتخاب شده با ϵ -greedy را در target وارد می‌کند. از نظر نظری انتظار می‌رود SARSA هزینه اکتشاف را بیشتر در مقادیر خود منعکس کند. با این حال، در نتایج واقعی این پروژه، میانگین ورود به خانه جریمه برای سیاست‌های حریصانه نهایی تقریباً یکسان است: ۰.۵۳۸۵ برای Q-Learning و ۰.۵۵۱۰ برای SARSA(۰.۷). میانگین برخورد با دیوار نیز ۲.۴۶۴۵ در برابر ۲.۴۴۵۵ است. بنابراین داده‌های این نقشه برای ادعای ایمن‌تر بودن SARSA کافی نیستند.

اثر on-policy بودن SARSA بیشتر در منفی‌تر شدن Q برخی حالت‌ها، دیرتر رسیدن به آستانه و توافق کمتر سیاست دیده می‌شود. کوچک بودن ϵ نهایی، استفاده از پاداش shaped یکسان و وجود یک مسیر غالب باعث شده تفاوت رفتار دو الگوریتم در اطراف جریمه‌ها طی ارزیابی حریصانه محدود باشد. این جمع‌بندی براساس training_episodes.csv، evaluation_episodes.csv و policy_mismatch_examples.json انجام شده است.

۱۳-۳- نیاز Value Iteration به مدل و محدودیت دو رویکرد

Iteration برای محاسبه امید ریاضی بلمن باید همه خروجی‌های ممکن هر زوج حالت-عمل، احتمال آن‌ها و پاداش متناظر را بداند. SARSA و Q-Learning فقط انتقال‌های نمونه‌گیری شده را مشاهده می‌کنند و بدون دسترسی به جدول P یاد می‌گیرند. در این پروژه، VI به دلیل فضای حالت کوچک همگرایی دقیق، سیاست مرجع و زمان اجرای کمی دارد؛ در مقابل، نیاز به مدل کامل و پیمایش همه حالت‌ها مهم‌ترین محدودیت آن است.

روش‌های مدل آزاد را می‌توان در محیط‌های ناشناخته به کار برد و مستقیماً از تجربه یاد گرفت، اما این مزیت با مصرف صدها هزار نمونه و حساسیت به ϵ ، α و Seed همراه است. همچنین ممکن است سیاست در حالت‌های کم‌بازدید کامل نشود. VI هنگام مقایسه آنلاین هیچ نمونه‌ای از محیط مصرف نمی‌کند، ولی به مدل صریح و Bellman backup نیاز دارد. بنابراین عبارت «صفر نمونه» اگر بدون اشاره به هزینه مدل بیان شود، تصویری ناقص از مقایسه ارائه می‌دهد.

۱۳-۴- بهترین λ از نظر سرعت و پایداری

$\lambda=0.9$ با میانگین ۹۱۶.۱ اپیزود سریع‌تر از سایر مقادیر به آستانه می‌رسد، اما return ارزیابی آن ۸۸.۲۷۳، SD میان Seedها ۱۴.۳۰۲ و میانگین گام ۳۱.۰۸۱۵ است. $\lambda=0.3$ پایدارترین return را دارد، ولی برای رسیدن به آستانه به ۱۱۲۷.۶ اپیزود نیاز دارد. $\lambda=0.7$ این آستانه را در ۹۵۹.۹ اپیزود رد می‌کند، return برابر با ۹۳.۸۹۹ و SD=۰.۱۴۹ دارد و با ۷۹.۸۲ درصد، بیشترین توافق سیاست SARSA را ثبت می‌کند. از این رو، $\lambda=0.7$ در این پروژه بهترین تعادل را فراهم کرده است، هرچند سریع‌ترین گزینه نیست.

۱۳-۵- سه حالت اختلاف سیاست

سه حالت ذخیره شده Q-Learning با بیشترین regret عبارت‌اند از $(۶,۱,۰)$ ، $(۷,۳,۰)$ و $(۸,۳,۱)$. در دو حالت نخست، Q-Learning عمل up را انتخاب می‌کند، در حالی که VI عمل down را با regret های ۵.۴۰۳ و ۵.۱۷۶ ترجیح می‌دهد. در حالت سوم، Q-Learning عمل down و VI عمل up را انتخاب می‌کند و regret برابر با ۴.۳۴۶ است. هر سه نمونه در شاخه‌ها یا راهروهایی قرار دارند که نسبت به مسیر اصلی بازدید کمتری داشته‌اند.

مقایسه Q آموخته شده با Q مدل نشان می‌دهد این اختلاف‌ها صرفاً ناشی از tie نیستند. برای مثال، در حالت $(۶,۱,۰)$ مدل مقدار down را ۵.۴۰۳ واحد بیشتر از up می‌داند، اما جدول یادگرفته شده up را بیش از ۱۰ واحد بالاتر از down برآورد کرده است. عامل همچنان از نقطه شروع مأموریت را کامل می‌کند، زیرا این خطاها مسیر غالب start-key-door-goal را قطع نمی‌کنند. جزئیات هر شش نمونه مربوط به Q-Learning و SARSA در policy_mismatch_examples.json قرار دارد.

۱۳-۶- مقایسه انتقال مشابه و متفاوت

در مقصد مشابه، full و همه β ها نرخ موفقیت اولیه ۱.۰ و return اولیه ۸۷.۷۴۹ دارند. در مقصد متفاوت، نرخ موفقیت اولیه آن‌ها ۰.۷۰۴ ± ۰.۰۱۴ و return اولیه - ۴۴۱.۶۸۳ است. scratch در هر دو مقصد با موفقیت اولیه صفر آغاز می‌شود. full برای رسیدن به آستانه در مقصد مشابه حدود ۵۹۳۳ و در مقصد متفاوت حدود ۵۹۴۳ نمونه مصرف می‌کند؛ این مقادیر برای scratch به ترتیب ۱۸۰۸۸ و ۱۸۲۴۰ نمونه هستند.

در پایان آموزش، همه روش‌ها نرخ موفقیت ۱.۰ دارند؛ با این حال، در مقصد متفاوت میزان توافق سیاست full فقط ۵۷.۷۴ درصد است، در حالی که scratch به ۶۹.۵۹ درصد می‌رسد. این اختلاف، همراه با دو شاهد انتقال منفی، نشان می‌دهد انتقال گسترده در محیط متفاوت شروع سریع‌تری ایجاد می‌کند، اما ممکن است بخشی از دانش نامناسب مبدأ را نیز حفظ کند. در محیط مشابه، $\beta=۰.۲۵$ با توافق ۷۳.۵۴ درصد بهترین کیفیت سیاست را در میان سناریوها دارد. در مجموع، انتقال در مقصد مشابه عمدتاً مثبت است، اما در مقصد متفاوت سود اولیه و خطای موضعی هم‌زمان دیده می‌شوند.

۱۴- محدودیت‌های مطالعه

۱۴-۱- محدودیت محیط و مقیاس

محیط این پروژه جدولی و کاملاً مشاهده‌پذیر است؛ بنابراین نتایج آن را نمی‌توان بدون بررسی بیشتر به مسائل پیوسته یا فضاهای حالت بزرگ تعمیم داد. پورتال‌ها ثابت‌اند و برای حفظ خاصیت مارکوف نیازی به متغیر حالت اضافی ندارند، در حالی که ویژگی‌هایی مانند مانع متحرک یا انرژی محدود می‌توانستند مسئله دشوارتری بسازند. همچنین فقط یک نقشه دانشجویی 16×16 بررسی شده است و استفاده از چند خانواده متفاوت نقشه، ارزیابی تعمیم‌پذیری را کامل‌تر می‌کرد.

۱۴-۲- محدودیت آماری

آزمایش‌ها با ۱۰ Seed انجام شدند و این تعداد همه عدم قطعیت آماری را از بین نمی‌برد. نواحی رنگی و نوارهای خطای شکل‌ها $\text{mean} \pm \text{SD}$ هستند، نه فاصله اطمینان. ارزیابی نهایی نیز شامل ۲۰۰ اپیزود است و شاید برای مشاهده رویدادهای بسیار نادر کافی نباشد. از طرف دیگر، Value Iteration فقط یک اجرای قطعی دارد؛ بنابراین SD صفر آن قابل مقایسه با پراکندگی روش‌های مدل‌آزاد در ۱۰ اجرای مستقل نیست.

۱۴-۳- محدودیت معیارها

مقادیر return در نسخه‌های sparse و shaped هم‌مقیاس نیستند. معیار حافظه نیز فقط مجموع بایت آرایه‌های جدولی ذخیره‌شده و بیشینه workspace مربوط به eligibility را دربر می‌گیرد و مصرف کامل RAM، سربار objectها یا حافظه Matplotlib را محاسبه نمی‌کند. زمان اجرا به سخت‌افزار، نسخه Python و بار لحظه‌ای سیستم وابسته است. policy agreement هم به قاعده شکستن تساوی با کوچک‌ترین index حساس است و حتی دو عمل با ارزش تقریباً برابر را به‌صورت کاملاً موافق یا مخالف دسته‌بندی می‌کند.

۱۴-۴- محدودیت انتقال

در انتقال انتخابی، برابری دقیق امضای محلی 3×3 به‌عنوان شرط انتقال استفاده شد. این معیار سخت‌گیرانه در مقصد مشابه فقط ۳۲ لایه و در مقصد متفاوت تنها ۲ لایه را منتقل کرد و در ارزیابی اولیه به هیچ موفقیتی منجر نشد. رویکردهای بعدی می‌توانند از شباهت نرم، تطبیق ویژگی‌ها یا انتقال محدود به حالت‌های پرتکرار استفاده کنند. همچنین روش full مقادیر مربوط به دیوارها و حالت‌های دسترس‌ناپذیر را نیز کپی می‌کند. این کار پیاده‌سازی را از نظر شکل آرایه ساده می‌سازد، اما مقدار transferred_state_count را بزرگ‌تر از تعداد حالت‌های واقعاً مفید نشان می‌دهد.

دو شاهد انتقال منفی فقط خطاهای محلی را نشان می‌دهند و به‌تنهایی شکست کلی روش را اثبات نمی‌کنند؛ به‌ویژه اینکه انتقال کامل همچنان عملکرد اولیه را بهتر کرده است. برای رسیدن به نتیجه‌ای عمومی‌تر، لازم است نقشه‌های مقصد بیشتری با شدت تغییر پیوسته بررسی شوند و آزمون‌های آماری جداگانه‌ای نیز انجام گیرد.

۱۵- جمع‌بندی

محیط نهایی همه اجزای اصلی مسئله کلید-در-هدف، عدم قطعیت عمل، خانه‌های جریمه، انتقال‌دهنده و پایان زمانی را در قالب یک MDP سازگار پیاده‌سازی می‌کند. افزودن has_key خاصیت مارکوف را حفظ کرده و اثر پورتال دوطرفه هم در انتقال واقعی محیط و هم در GUI قابل مشاهده است. پاداش potential-based shaping نیز سیاست بهینه را در ۱۶۸ حالت تغییر نداد، هرچند بهبود سرعت Q-Learning نسبت به پاداش sparse در این نقشه محدود بود.

در این فضای کوچک، Value Iteration یک مرجع دقیق و سریع فراهم کرد. کاهش نمایی ϵ در Q-Learning زودتر به موفقیت رسید، اما کاهش خطی به توافق سیاست بیشتری دست یافت. در SARSA، افزایش λ انتشار پاداش به تصمیم‌های قبلی را سریع‌تر کرد، ولی $\lambda=0.9$ ناپایداری بیشتری نشان داد و $\lambda=0.7$ نتیجه متعادل‌تری داشت. هر سه روش در ارزیابی نهایی نرخ موفقیت ۱۰۰ کسب کردند، اما تفاوت‌های زمان، تعداد نمونه، حافظه و توافق سیاست اطلاعات بیشتری از نرخ موفقیت نهایی ارائه دادند.

انتقال یادگیری در مقصد مشابه از همان ارزیابی اولیه موفق بود و در مقصد متفاوت نیز تا حدی سودمند باقی ماند. باین‌حال، انتقال کامل در مقصد متفاوت توافق سیاست کمتری از scratch داشت و دو خطای موضعی با regret دقیق شناسایی شد. آموزش در محیط مقصد هر دو خطا را اصلاح کرد. جمع‌بندی کلی این است که ارزیابی با یک معیار واحد کافی نیست و باید عملکرد اولیه، سرعت یادگیری، نتیجه نهایی، کیفیت سیاست و موارد انتقال منفی به‌طور جداگانه بررسی شوند.

۱۶- ساختار تحویل و الزامات مخزن

مطابق صورت پروژه، مخزن نهایی باید به صورت public و با نام RL_FinalProject_۴۰۴۰۰۳۵۶ ایجاد شود. بهتر است شاخه اصلی main باشد و دست کم سه commit معنادار، روند توسعه محیط، الگوریتم‌ها، آزمایش‌ها و GUI را نشان دهند. از آنجا که تاریخچه commit در ارزیابی پروژه بررسی می‌شود، قرار دادن همه فایل‌ها در یک commit نهایی این شرط را برآورده نخواهد کرد.

جدول 22 - چک‌لیست تحویل نهایی

مورد	محل یا نام	وضعیت/اقدام لازم
گزارش نهایی	در ریشه report.pdf	این Word پس از درج ۱۵ شکل به PDF تبدیل شود
کد کامل	environments, agents, transfer, gui, experiments	موجود در ۷۳
داده خام	results/raw_data/*.csv,json,jsonl	موجود و hash شده
مدل‌ها	results/models/*.npz	۱۹۶ فایل موجود
محتوای بصری	results/figures/*.png	۱۴ شکل موجود؛ اسکرین‌شات GUI افزوده شود
README	README.md	نصب، اجرا و بازتولید موجود
وابستگی‌ها	requirements.txt	موجود
تست‌ها	tests/	۴۵ تست موجود و پاس شده
GitHub	public: RL_FinalProject_۴۰۴۰۰۳۵۶	پیش از تحویل ایجاد/کنترل شود
Commitها	حداقل ۳ commit معتبر	پیش از تحویل کنترل شود

فایل‌های وابسته به رایانه شخصی، محیط مجازی، cache، لاگ‌های پذیرش و ZIP نسخه‌های قبلی نباید وارد مخزن نهایی شوند. دستورهای اصلی python main.py و python experiments/run_experiments.py نیز باید مستقیماً از ریشه مخزن اجرا شوند. پیش از تحویل، تمام کادرهای «محل درج شکل» باید با تصویر متناظر جایگزین شوند و caption و تحلیل مربوط به هر شکل در جای خود باقی بماند.

پیوست

در فرایند انجام پروژه، از ابزار هوش مصنوعی برای پیشنهاد اولیه و کلی ساختار کد به جهت بهینه بودن، یافتن خطاها، طراحی UI استفاده شد. با این حال، هیچ نتیجه عددی بدون اجرای واقعی کد وارد گزارش نشده است. هر پیشنهاد با معادلات، تست‌ها و داده‌های خام بررسی و در صورت نیاز اصلاح شد. جدول زیر چند نمونه از پیشنهادهای اولیه و تغییراتی را نشان می‌دهد که پس از بررسی فنی روی آن‌ها انجام گرفت.

جدول 24 - موارد استفاده از ابزار کمکی

مورد استفاده	پیشنهاد اولیه	تغییر انجام‌شده	دلیل اصلاح
طراحی پاداش	پاداش مستقیم برای نزدیک شدن به کلید/هدف	استفاده از potential-based shaping و کنترل توافق ۱۰۰٪	پاداش فاصله خام می‌توانست سیاست یا حلقه را تغییر دهد
ثبت trace SARSA	ذخیره فقط E زوج جاری	ذخیره همه trace های فعال قبل و بعد از decay	پیشنهاد اولیه انتقال خطا به حالت‌های قبلی را اثبات نمی‌کرد
تحلیل انتقال	final success مقایسه	افزودن initial success ، نمونه تا آستانه، توافق و regret	final success همه روش‌ها ۱۰۰ و به‌تنهایی گمراه‌کننده بود
انتقال انتخابی	کپی بر اساس یکسان بودن نوع خانه	برابری امضای محلی ۳×۳ و گزارش تعداد منتقل‌شده	نوع خانه به‌تنهایی تغییر همسایگی و عمل مناسب را نشان نمی‌داد
اعتبارسنجی	اکتفا به اجرای بدون crash	۴۵ تست، hash ، verifier و بازپخش مدل‌ها	اجرای موفق لزوماً صحت معادله و داده را ثابت نمی‌کند